

ZLang HDL

Language Reference

Community Release

v0.1.0a24

Syntax, exact types, state, timing, protocols and verification contracts

Apache-2.0 Community distribution

Contents

1 Why ZLang HDL . . . 2
1.1 What makes ZLang different . . . 2
1.2 From intent to hardware . . . 3
1.3 Core principles . . . 3
1.4 Quick start: one complete design . . . 3
2 Types, numerics, and packed representation . . . 4
2.1 Characters, fixed strings, and tuples . . . 4
2.2 Nominal enums . . . 5
2.3 Nominal tagged unions . . . 6
2.4 Scalar families . . . 6
2.5 Integer width rules . . . 6
2.6 Fixed point . . . 7
2.7 Vectors and structs . . . 8
2.8 Slicing, concatenation, and representation . . . 9
2.9 Current boundaries . . . 11
3 Expressions, functions, and generics . . . 11
3.1 Operators and selection . . . 11
3.2 Aggregate value ergonomics . . . 12
3.3 Pure functions . . . 13
3.4 Compile-time integer modulo . . . 14
3.5 Compile-time functions and selection . . . 14
3.6 Functional datapath . . . 15
3.7 Typed bit and collection composition . . . 16
3.8 Generic structs, functions, and operators . . . 17
4 Compile-time integer modulo . . . 17
5 Sequential state, rules, pipelines, and storage . . . 18
5.1 Clock and reset . . . 18
5.2 Registers and next state . . . 19
5.3 Delays and fixed pipelines . . . 22
5.4 Exact module timing contracts . . . 22
5.5 Guarded atomic actions . . . 22
5.6 FIFOs . . . 24
5.7 Writable memories . . . 25
5.8 Initialized synchronous ROMs . . . 29
5.9 Stateful hierarchy . . . 30
6 Physical clocks and resets . . . 30
6.1 Recommended asynchronous reset . . . 30
6.2 Backend lowering . . . 31
6.3 Hierarchy inheritance . . . 31
6.4 Formal applicability . . . 32
6.5 Boundaries . . . 32
7 Hierarchy, protocols, and explicit CDC . . . 33
7.1 Modules and instances . . . 33
7.2 Clock and reset boundary . . . 35
7.3 Public top ABI . . . 35
7.4 Ready/valid . . . 35
7.5 Credit and request/response . . . 35
7.6 Aggregate protocols and the standard library . . . 36
7.7 Packets, virtual channels, and arbitration . . . 36
7.8 CSR blocks . . . 37
7.9 Clock-domain crossings . . . 38
8 Named module interfaces . . . 39
8.1 Exact conformance . . . 39
8.2 Aggregate protocol signatures . . . 40
8.3 Selection and backend identity . . . 40
8.4 Current boundaries . . . 41
9 Tagged unions . . . 41
9.1 Type and layout . . . 41

9.2 Construction and matching . . . 41
9.3 Deliberate boundaries . . . 41
10 Parameterized aggregate protocols . . . 42
11 Implementation intent, frequency, and II . . . 42
11.1 `intent` clauses . . . 43
11.2 Frequency control and synthesis trials . . . 44
11.3 Ready/valid contracts and II . . . 45
12 Optimization and formal verification . . . 46
12.1 Canonical and selected IR . . . 46
12.2 One implementation-policy path . . . 46
12.3 First-class verification goals and contracts . . . 50
12.4 Formal layers . . . 52
12.5 Reports . . . 53
13 E-graph and optimization responsibilities . . . 53
14 Storage-owning instance arrays . . . 54
14.1 Supported boundary . . . 54
15 Source and product identity . . . 55
16 Language support matrix . . . 55
16.1 Language and execution . . . 55
16.2 Formal coverage . . . 56
17 Known limitations . . . 57
17.1 Language and RTL boundaries . . . 57
17.2 Other bounded combinations . . . 58
17.3 Verification boundaries . . . 58
18 ZLang HDL concise source-authoring reference . . . 58
18.1 Non-negotiable rules . . . 58
18.2 Smallest valid modules . . . 59
18.3 Types, literals, and exact conversions . . . 60
18.4 Aggregates and functions . . . 61
18.5 State, hierarchy, and protocols . . . 61
18.6 Functional datapath and implementation intent . . . 62
18.7 Verification . . . 63
18.8 Commands and completion check . . . 63
Font license notice . . . 65

Syntax, exact types, state, timing, protocols and verification contracts.

1 Why ZLang HDL

ZLang HDL is a typed hardware description language and compiler for designs whose numerical, state, timing, and protocol meaning must remain explicit. The production backend emits Direct SystemVerilog from checked semantics. Unsupported combinations are errors; they are not silently reinterpreted as different hardware.

1.1 What makes ZLang different

Concern	ZLang approach
Widths	Exact; no silent resize
Fixed point	Explicit quantization
State	Explicit next-state and atomic actions
Timing	Exact latency contracts
CDC	Explicit semantic crossing
Implementation choice	`implement` plus `intent`
Optimization	Exact legal alternatives only
Physical mapping	Target- and resource-aware
Verification	Evidence status is retained
RTL	Direct-SV from typed semantics

1.2 From intent to hardware

```
Designer intent
      ↓
Exact typed semantics
      ↓
Exact implementation alternatives
      ↓
Target/resource planning
      ↓
Verification and evidence
      ↓
Direct SystemVerilog
      ↓
FPGA / ASIC toolchain
```

Source defines required behavior. The compiler may vary structure only through exact alternatives admitted by the language and selected profile. Timing and state contracts remain part of the design. Resource estimates, synthesis measurements, routed measurements, bounded checks, and proofs retain distinct evidence labels.

1.3 Core principles

- Width and signedness changes are explicit with operations such as `extend`, `truncate`, and `bitcast`.
- Fixed-point rounding and overflow are explicit at `quantize` boundaries.
- Clock, reset, domain ownership, state updates, and protocol transfers have exact cycle meaning.
- Rules commit atomically from one pre-edge snapshot.
- Protocol adaptation and clock-domain crossing are explicit.
- Compilation and published artifacts are deterministic for the same inputs.
- Unsupported forms stop with a diagnostic.

1.4 Quick start: one complete design

The following design combines exact arithmetic, explicit state, an atomic rule, a safety property, and bounded implementation selection.

```
module RunningSum {
    clock clk reset rst
    in enable : bit
    in a, b : u8
    out reg total : u10 = 0
    out pair_sum : u9

    pair_sum = implement {
        a + b
        intent { latency == 0 ii == 1 minimize lut }
    }

    accumulate: when enable {
        total <- truncate<10>(total + extend<10>(a + b))
    }

    assert pair_sum_is_exact @ clk {
        extend<10>(pair_sum) == extend<9>(a) + extend<9>(b)
    }
}
```

In `pair_sum = ...`, the `=` assignment defines a combinational connection; it does not update a register. Addition produces its exact result width, so two `u8` operands produce `u9`. The state

update explicitly extends the sum and truncates the accumulated result to ten bits: `total` wraps modulo 1024 rather than reporting overflow. The `<-` update schedules the register's next value at the active clock edge. The rule reads one pre-edge snapshot and commits atomically. The `out reg` declaration creates architectural storage that holds its value when no update occurs.

The expression inside `implement` defines the required value. Its `intent` constrains legal implementations without changing that value. Here the result is combinational (`latency == 0`) with initiation interval one (`ii == 1`). The assertion checks the selected `pair_sum` against an independently widened addition; it is not an overflow check on the wrapping accumulator.

1.4.1 Source files and notation

A source file may contain imports, type aliases, structs, pure functions, operator overloads, rewrite declarations, target/resource declarations, and multiple modules. Select a top explicitly with `--top` when needed. Names are case-sensitive; there are no semicolons.

Form	Meaning
`name = expression`	Define a value, combinational connection, or resource control, according to the declaration.
`state <- expression`	Schedule next state at the active edge.
`source -> destination`	Connect typed endpoints or bind a command.
`key => expression`	Select a value arm.
`thing @ clk`	Associate a declaration with a clock domain.
`object.field`	Select an aggregate, protocol, or resource member.

2 Types, numerics, and packed representation

ZLang types describe hardware representation exactly. Width, signedness, fixed-point scale, vector length, struct identity, and fixed-point overflow policy participate in typing and canonical identity.

2.1 Characters, fixed strings, and tuples

`char` is a source alias for canonical `u8`; it is intentionally not a separate nominal type. `string<N>` is a source alias for the corresponding fixed hardware collection and normalizes to canonical `vec<N,u8>`:

```
letter : char = 'A'
tag : string<4> = "OFDM"
joined = concat("802.", "11a")
```

Literals contain printable ASCII directly and may use `\0`, `\n`, `\r`, `\t`, `\\`, `\'`, `\"`, or an exact `\xNN` byte. Strings have no hidden terminator or length field and cannot be empty. They inherit vector indexing, equality, `length`, `generate`/`map`, homogeneous concatenation, pure functions, generic specialization, and typed compile-time constant parameters. Registers, FIFOs, synchronous memories, and ROMs store them with the same semantics as `vec<N,u8>`. When packed, the first character is the least-significant byte (`pack("AB") == 0x4241`). Unicode, interpolation, formatting, padding, and dynamic-length strings are not implicit operations.

Tuples are structural ordered aggregates. Tuple types use `(T,U)` and tuple values use the same positional spelling:

```
pair : (u8, bit) = (data, last)
payload = pair[0]
(next_data, next_last) = pair
```

Tuple arity is 2 through 8. Projection is a zero-based integer literal and flat destructuring introduces exhaustive immutable bindings. Nested tuple values are legal, but nested/wildcard patterns, runtime tuple selection, tuple update, arithmetic, and ordering are not. Exact `==`/`!=` is

structural. Component zero occupies the least-significant packed region; `pack`/`bitcast` require every component to satisfy the ordinary non-enum packing rules.

`char` is indistinguishable from `u8` during overload resolution, and `string<N>` is indistinguishable from `vec<N,u8>`. Tuple identity, by contrast, is the recursive ordered sequence of component types.

2.2 Nominal enums

Enums give control state a nominal hardware type without exposing a numeric encoding in source:

```
enum State { Idle Header Payload Done }

reg state : State = State.Idle
active = state == State.Payload
```

Members use qualified names. Their declaration-order ordinals are encoded in `max(1, ceil_log2(member_count))` bits, so a one-member enum still occupies one bit. The encoding is a stable storage/backend ABI, but enum values are not integers: arithmetic, ordering, resizing, and cross-enum comparison are errors. Equality and inequality require the same nominal enum declaration.

When an external encoding is part of the hardware contract, declare an exact `bit`, `bits<W>`, or unsigned `uN`/`u<W>`/`uint<W>` representation. Omitted member values start at zero and otherwise continue from the preceding resolved value:

```
enum StatusCode : u8 {
    Idle
    Started = 10
    Waiting = 120
    Complete
}
```

This assigns `0`, `10`, `120`, and `121`. Every resolved code must be unique and fit the declared backing type; `enum Flag : bit { Clear Set }` is therefore limited to two values. Declaration order still controls exhaustive source selection, while the resolved code is the physical representation. The unbacked ordinal form above remains unchanged.

Raw boundaries use total, typed operations rather than enum casts:

```
raw     : bits<3> = enum_encode(rate)
valid   : bit = enum_valid<WifiRate>(raw)
decoded : WifiRate =
    enum_decode<WifiRate>(raw, WifiRate.Continue)
```

`enum_encode` is lossless and returns the enum's exact declared backing type (`bit`, `bits<W>`, or `uN`/`u<W>`/`uint<W>`); an unbacked ordinal enum retains its minimal `bits<W>` representation. `enum_valid<T>` and `enum_decode<T>` require that same exact representation. Decode returns its required same-enum fallback for every hole, so an invalid representation never becomes an invalid nominal value. These operations perform no resize.

Enum values remain nominal and cannot participate directly in arithmetic. An unsigned-backed enum deliberately exposes a numeric boundary through `enum_encode`, so `enum_encode(status) + 1` uses the normal widened unsigned arithmetic rules. A `bit`- or `bits<W>`-backed enum requires an explicit `bitcast<uW>` before arithmetic; there is no implicit raw-bits-to-integer cast.

Enums may appear in locals, outputs, registers/rules, structs, and vectors. A top-level enum input is rejected because an external invalid bit pattern would violate the nominal value invariant. General `unpack<Enum>` is likewise deferred. Exhaustive selection is described under operators and selection.

2.3 Nominal tagged unions

A tagged union represents one of several named payload shapes without exposing raw tag arithmetic:

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }
result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

The nominal declaration identity is part of the type. The source-order tag is stored in the most-significant bits; fields occupy the maximum-width payload in source order and shorter payloads receive zero low-order padding. A fieldless constructor is written `Message.Idle`. Every pure `match` is exhaustive, names each variant once, and binds exactly the declared fields. See the complete bounded surface.

2.4 Scalar families

Spelling	Meaning
`bit`	A one-bit control value, distinct from integers and raw bits.
`uN`, `u<N>`, `uint<N>`	An `N`-bit unsigned integer.
`sN`, `sint<N>`	An `N`-bit two's-complement signed integer.
`bits<N>`	An `N`-bit raw vector without integer arithmetic.
`fixed<W,F>`	Signed fixed point, `W` total bits and `F` fractional bits, wrapping target narrowing.
`ufixed<W,F>`	Unsigned fixed point with wrapping target narrowing.
`fixed_sat<W,F>`	Signed fixed point with saturating target narrowing.
`ufixed_sat<W,F>`	Unsigned fixed point with saturating target narrowing.

Widths are positive compile-time integers. Fixed-point types additionally require `0 <= F < W`. The families are distinct: no implicit signed/unsigned, integer/raw-bit, or integer/fixed conversion is performed.

Aliases disappear during typing:

```
type Address = uint<32>
type Payload = bits<64>
```

Alias cycles, duplicate declarations, and redefinition of built-ins are errors.

2.5 Integer width rules

An unsuffixed integer literal has the smallest exact hardware type implied by its value. Non-negative literals are unsigned; syntactic negative literals are signed two's-complement values:

Literal	Inferred type
`0`, `1`	`u1`
`2`, `3`	`u2`
`255`	`u8`
`-1`	`s1`

Literal	Inferred type
`-2`	`s2`
`-3`, `-4`	`s3`
`-123`	`s8`

Decimal, hexadecimal, and binary spellings with the same value infer the same type; leading zeros do not request storage width. A direct literal at an explicitly typed assignment, field, argument, or return boundary instead uses that target type when the value fits exactly. An out-of-range literal is an error. Context never recursively resizes a compound expression, so `concat(0,1)` and `0 + 1` retain the types derived by their own operators.

Unary `-` applied to a non-literal remains ordinary typed arithmetic. It does not reinterpret an unsigned signal as a signed one.

Expression	Result
`uN + uM`	`u(max(N,M)+1)`
`sN + sM`	`s(max(N,M)+1)`
`uN - uM`	`u(max(N,M))`, modular underflow
`sN - sM`	`s(max(N,M)+1)`
same-family `N * M`	Width `N+M`
`&`, `|`, `^`	Matching family, widest operand width
shifts	The left operand's exact type and width
comparisons	`bit`

Addition and multiplication preserve carry bits. A left shift discards bits beyond its fixed result width. Signed right shift is arithmetic; unsigned and raw-bit right shift is logical.

Use explicit resizing:

```
wide = extend<17>(value)
low  = truncate<8>(wide)
```

`extend<N>` preserves signedness behavior and `truncate<N>` keeps low bits. Neither changes the type family.

At an explicitly typed output, local, register write, storage write, child binding, struct field, or declared function-return boundary, the width may be obtained from that boundary while the conversion remains explicit:

```
low : u8 = truncate(wide)
wide_again : u17 = extend(low)
```

The contextual and explicit-width forms normalize to the same typed resize. An inferred local, a nested arithmetic operand, or a mismatched type family has no such context and must keep `truncate<N>` or `extend<N>`.

2.6 Fixed point

Concrete spellings expose integer and fractional widths directly:

Spelling	Canonical type	Narrowing policy
`SF8.8`	`fixed<16,8>`	wrap
`SF_Sat8.8`	`fixed_sat<16,8>`	saturate
`UF8.8`	`ufixed<16,8>`	wrap
`UF_Sat8.8`	`ufixed_sat<16,8>`	saturate

For signed values the integer part includes the sign bit. `SF8.8` therefore represents `-128.0` through `127.99609375` in steps of `2^-8`.

Arithmetic is widened before any storage conversion. Fixed addition and subtraction require equal fractional widths; multiplication returns total width `W1+W2` and fractional width `F1+F2`. Target overflow policy applies only at an explicit or contextual storage conversion, not after every intermediate node.

Direct fixed-point literals must be exactly representable and in range, even for a saturating target. Loss is explicit:

```
y = quantize<SF_Sat8.8>(value) {
    round nearest_even
    overflow saturate
}
```

The rounding mode is mandatory; there is no implicit rounding default.

`round` mode	Discarded fractional part
`toward_zero`	Truncate toward zero.
`floor`	Round toward negative infinity.
`away_zero`	Round away from zero if any fractional part remains.
`nearest_even`	Round to nearest; on an exact tie choose the even stored integer.

Form	Target and overflow behavior
`quantize(value, MODE)`	Target comes from an unambiguous fixed-point assignment/return context; overflow defaults to that target type's policy.
`quantize(value) { round MODE overflow POLICY }`	Contextual target; both clauses are required.
`quantize<T>(value) { round MODE overflow POLICY }`	Explicit fixed-point target; both clauses are required.

`POLICY` is `wrap` (modular truncation to the target bit width) or `saturate` (clamp to the target's minimum/maximum). Ordinary `fixed`/`ufixed` targets default to `wrap`; `fixed_sat`/`ufixed_sat` targets default to `saturate` only in the short two-argument form. The block's explicit policy takes precedence. Conversion order is exact rescale, rounding, then overflow. `fixed_raw(0x0C22)` creates a target-context raw pattern; `fixed_to_raw` exposes the stored integer.

An exact fixed-point `dot(a,b)` retains the full accumulator. If the destination reduces fractional precision, give one post-accumulation rounding mode:

```
out y : SF_Sat8.8
y = dot(coefficients, samples, floor)
```

2.7 Vectors and structs

`vec<N,T>` has a compile-time length:

```
in samples : vec<4,u8>
```

An explicitly typed boundary can use a non-empty vector literal or contextual replication:

```
pair          : vec<2,u8> = [left, right]
zero_values   : vec<4,u8> = repeat(0)
zero_values_2 : vec<4,u8> = repeat<4>(0)
```

Every element must have the exact contextual element type. `repeat(value)` requires the destination vector to provide its length; `repeat<N>(value)` makes the length explicit. Both forms retain ordinary vector sequence order and lower to the existing typed generation representation.

Constant indexes are checked during elaboration. Runtime reads are accepted only when conservative range analysis proves every encoded value in range; ZLang does not silently mask or clamp. A rule may update one range-proven element of a one-dimensional `reg vec<N,T>` atomically. Nested paths, multiple dynamic writes to the same register action group, and runtime-selected instances remain unsupported; see Sequential logic, rules, and storage.

Structs are nominal products:

```
struct Packet {
    data : u64
    last : bit
}

packet = Packet { data = payload last }
```

Constructors are complete and may use same-name field punning. An immutable update rebuilds the same nominal struct and preserves fields not named in the update:

```
completed = packet with { last = 1 }
```

Unknown or duplicate fields and values of the wrong exact field type are rejected. This is a pure value expression, not mutation. Recursive layouts are still rejected. Generic structs are covered in Expressions, functions, and generics.

Structural `==` and `!=` are available for exact matching struct and vector types. Comparison is recursive over every field/element and returns `bit`; it does not perform conversion, resizing, or nominal coercion.

An immutable nominal struct can also be destructured exhaustively:

```
Packet { data, last } = packet
```

Every declared field must appear exactly once, under its declared name, and the new immutable bindings may not shadow another symbol. Destructuring lowers to ordinary typed field values; it does not add tuple patterns, partial patterns, renaming, mutation, or runtime matching.

2.8 Slicing, concatenation, and representation

An inclusive, compile-time bit slice returns raw bits:

```
in word : u16
out byte : bits<8>
byte = word[15:8]
```

Both `MSB` and `LSB` are compile-time integer expressions. The result is `bits<MSB-LSB+1>`. Reversed, negative, or out-of-range bounds are errors; slicing never resizes, reinterprets, or clamps its operand.

A `bits<N>` value also supports concise single-bit selection:

```
in raw : bits<24>
out lsb : bit = raw[0]
out msb : bit = raw[23]
```

Packed indices use conventional bit numbering: index zero is the least-significant bit, exactly as `[0:0]`. A runtime unsigned selector is legal only when its compiler-proven interval lies entirely within the packed width; there is no implicit modulo, clamp, or out-of-range hardware behavior. A fixed-width runtime window uses LSB-based `+:` spelling:

```
in raw    : bits<16>
in offset : u3
out byte  : bits<8>
byte = raw[offset +: 8]
```

`WIDTH` is a positive compile-time integer. The compiler requires `offset + WIDTH <= packed_width` for every possible offset. Vector element zero likewise occupies the least-significant packed element region.

`concat(a,b,...)` accepts at least two operands. When every operand is a vector with the exact same element type it returns one longer vector, preserving source and element order:

```
in left  : vec<2,u8>
in right : vec<3,u8>
out all  : vec<5,u8>
all = concat(left, right) // left[0], left[1], right[0], right[1], right[2]
```

Otherwise no operand may be a vector and all operands must be recursively bit-packable. The result is raw `bits<N>` and the first operand occupies the most-significant bits:

```
joined = concat(high, low) // high is the MSB portion
```

Each scalar operand is typed independently, before the result width is known. For example, `concat(0,1)` is `bits<2>` and `concat(3,0)` is `bits<3>`. Neither an assignment target nor a function caller may widen an operand or the completed concatenation implicitly.

Exact packed fill constants are available without manufacturing a numeric literal of the desired width:

```
clear_mask : bits<24> = zeros<24>
set_mask   : bits<24> = ones<24>
```

The width is a positive compile-time integer expression. Both forms produce exact `bits<N>` values and lower to ordinary constants; they are not vector replication. The separate `zero<x>`/`ones<x>` spellings inside an `equiv` declaration remain typed pattern constants rather than hardware expressions.

Vector/scalar mixtures and different vector element types are errors. Convert a vector explicitly with `bitcast<bits<W>>` when raw bit assembly is intended.

`reshape(value)` obtains a vector target shape from an explicitly typed context; `reshape<T>(value)` supplies it directly. Reshape recursively enumerates leaves outer-to-inner, then rebuilds the requested shape without packing them:

```
in matrix : vec<2,vec<4,u8>>
out flat  : vec<8,u8>
flat = reshape(matrix)
matrix_again = reshape<vec<2,vec<4,u8>>>(flat)
```

Source and target must be vectors with the same leaf count and exact same leaf type. Reshape has no endianness and performs no numerical conversion.

`bitcast<T>(value)` exposes or reconstructs the exact stored representation when source and target have equal packed width and both are recursively bit-packable and non-enum. It never resizes, sign-extends, truncates, rescales, rounds, or saturates:

```
raw    : bits<8> = bitcast<bits<8>>(signed_value)
signed : s8      = bitcast<s8>(raw)
```

`pack(value)` and `unpack<T>(raw)` remain supported low-level compatibility spellings and normalize to the same typed `Bitcast` operation. Signed integers and signed fixed-point values use their

stored two's-complement pattern.

Aggregate layout is deterministic across simulation and generated RTL:

- struct field zero (declaration order) is at the MSB;
- vector element zero is at the LSB;
- tuple component `item0` is at the LSB;
- nested vectors and tuples apply the LSB-first rule recursively;
- nested structs retain declaration-order/MSB-first field layout.

The initialized `rom<T,N>` image format uses this same layout for each word. Images contain one exact-width binary word per line with address zero first; they do not inherit host byte order. Consequently simulator values and Direct-SV `$readmemb` agree on nested aggregate and fixed-point ROM contents. See Initialized synchronous ROMs.

Scalar numeric/raw types and recursively bit-packable structs, vectors, and structural tuples are accepted. Nominal enums, including an aggregate containing an enum, are intentionally rejected by `bitcast`, `pack`, and `unpack`. Enum ordinal storage remains available through ordinary typed values and `reshape`, but raw construction could create an invalid ordinal; no validity policy for that conversion is defined.

At explicitly typed assignment/storage boundaries, ZLang may insert an equal-width raw bitcast only when either source or target is `bits<N>` or flat `vec<N,bit>`. This makes raw wiring concise while preserving the rule that direct signed-to-unsigned, integer-to-fixed, resizing, and rescaling conversions are never implicit. Raw conversion does not participate in function arguments, operator overloads, generic inference, switch branch unification, or protocol compatibility.

Scalar `reduce(&, value)`, `reduce(|, value)`, and `reduce(^, value)` reduce the stored bits of `bit`, `bits<N>`, `uN`, or `sN` and return `bit`. `parity(value)` is the concise XOR form and also accepts `vec<N,bit>`. Existing vector reduction still reduces elements and returns the element type; fixed-point and struct scalars are not representation-bit reduction operands.

2.9 Current boundaries

Packed-bit selection and fixed-width `value[offset +: WIDTH]` accept an unsigned runtime index only when its complete proven range fits the source. Runtime `MSB:LSB` bounds, variable-width, reversed or out-of-range slices, zero or unresolved packed widths, one-operand or heterogeneous-vector concatenation, runtime reshape, unequal-width bitcast, enum bitcast, and external top enum inputs remain deliberately rejected. These boundaries are diagnosed rather than inferred by an output format. See Known limitations.

3 Expressions, functions, and generics

ZLang expressions are pure typed hardware values unless they explicitly contain a timing construct. Assignment requires an exact canonical target type.

3.1 Operators and selection

From highest to lowest precedence:

1. calls, parentheses, aggregate selection, conversions, functional forms;
2. unary `-`, logical `!`, and bitwise `~`;
3. `*`, and compile-time `/` and integer `%` (left-associative);
4. `+`, binary `-`;
5. `<<`, `>>`;
6. `&`, then `^`, then `|`;
7. `==`, `!=`, `<`, `<=`, `>`, `>=`;
8. `&&`, then `||` in compile-time/equivalence guards;
9. right-associative `condition ? true_value : false_value`.

Unary `-` is available for numeric scalar/fixed types and may be overloaded for a nominal struct. It is not a general implicit-cast mechanism. Runtime logical `!` accepts exactly `bit` and returns `bit`; it is equivalent to an exact comparison with zero. `/`, `&&`, and `||` are available only where the compile-time/equivalence sublanguage accepts them; they do not infer runtime divider or

general truthiness for multi-bit hardware values. Integer `%` is likewise compile-time only; it never infers a runtime remainder. Bitwise `~` accepts `bit`, raw bits, and signed/unsigned integers, preserving their exact type and width. It is not fixed-point negation.

Two-way runtime selection can use `?:` or `mux`:

```
y = select ? a : b
y = mux(select, a, b)
```

The condition is `bit` and both arms have one exact type. Numeric `switch` requires unique fitting keys and an `else` arm:

```
y = switch opcode {
    0 => a
    1 => b
    else => 0
}
```

A switch over a nominal enum uses qualified members and must cover every member. It needs no catch-all arm, which keeps a newly added state from silently taking an old default path:

```
next = switch state {
    State.Idle => State.Header
    State.Header => State.Payload
    State.Payload => State.Done
    State.Done => State.Idle
}
```

Numeric keys in an enum switch, members from another enum, duplicate members, and incomplete coverage are errors. Numeric switches retain their existing mandatory `else` behavior.

Nominal tagged unions use the parallel exhaustive `match` expression. Each arm names its union and variant and binds the exact source-declared payload names; there is no default arm or hidden priority:

```
y = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => bitcast<u8>(extend<8>(code))
}
```

Typing lowers this surface to an ordinary exact-width `switch` over the union tag plus typed field projections before canonical optimization and backend emission.

Exact matching structs, vectors, and structural tuples also support `==` and `!=`. These comparisons recursively combine the ordinary exact leaf comparisons and return `bit`; mismatched nominal structs, vector lengths, tuple component order/arity, or element types are errors.

3.2 Aggregate value ergonomics

Vector literals and replication use their explicit destination type:

```
pair        : vec<2,u8> = [left, right]
zero_values : vec<4,u8> = repeat(0)
```

`repeat<N>(value)` may state the compile-time length explicitly. The contextual and explicit forms lower to the same existing vector-generation semantics. Struct update is likewise a pure expression over an existing nominal value:

```
next = current with { valid = 1 payload = replacement }
```

It preserves omitted fields and lowers to a complete typed struct construction; it does not create mutable records or a backend-specific update operation. Exhaustive `Struct { field, ... } = value` destructuring similarly lowers to ordinary immutable field bindings. It requires every field exactly once and does not provide partial patterns, renaming, or runtime pattern matching. Flat tuple destructuring `(first, second) = pair` likewise evaluates one RHS and introduces fresh immutable projections. Nested, wildcard, and rest patterns remain outside this bounded surface.

3.3 Pure functions

Functions are top-level, pure, and combinational. The smallest body is one result expression:

```
fn tap(sample : u8, coefficient : u8) -> u16 {
    sample * coefficient
}
```

They may call other non-recursive pure functions. They cannot capture module ports or contain state, clocks, delays, protocols, or pipelines. Return types may be explicit or inferred for both ordinary and generic functions:

```
fn make_tag(high : bits<2>) {
    concat(high, zeros<2>, ones<2>)
}
```

An inferred body is checked without a caller-provided expected type, and its exact final-expression type becomes the function signature. Forward references are resolved independent of declaration order. Direct or indirect inferred return cycles are rejected; ZLang does not attempt a recursive type fixed point. For ordinary functions, explicit and inferred spellings with the same exact signature lower to the same callable identity. Generic specialization identity continues to include the generic declaration form and its exact specialization arguments.

A body may introduce source-ordered immutable bindings without `let`, `const`, or `return`; the final expression is the result:

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Each binding has its exact inferred type, may reference parameters and earlier bindings, and cannot shadow or be reassigned. Forward references and module captures are rejected. This syntax is normalized into the same concrete typed expression/call IR as the nested expression spelling.

Source units are declaration-order independent: functions, structs, enums, and modules may be declared after their users, and a shipped library unit may contain declarations without a hardware top. Parameterized modules can state compile-time obligations at specialization:

```
module Queue<type T,D=4>
where D >= 2 && is_power_of_two(D) {
    // ...
}
```

The constraint is checked against exact type/value arguments and creates no runtime hardware.

3.3.1 Typed constants and statically selected functions

Module and generic-function specializations may carry required named compile-time constants and pure function references:

```
fn widen(x : u8) -> u9 { extend<9>(x) }

fn transform_with<
    type A,
    type B,
    operation : fn(A) -> B
>(x : A) {
    operation(x)
}

module ImageBank<type T,N,IW,image : vec<N,T>> {
    clock clk
    reset rst
    in address : uint<IW>
    out data : T

    rom table : rom<T,N> { read_latency 1 init image }
    table.read_address = address
    data = table.read_data
}
```

These argument kinds are always named: `image=image` and `operation=fn widen`. A generic target may be selected explicitly, for example `operation=fn widen_as<A=u8,B=u9>`. Constants must have the exact declared, recursively bit-packable non-enum type and must evaluate completely during elaboration. A function reference must be pure and match the exact parameter and return types; it is not a runtime function pointer and cannot be stored, returned, compared, or carried through hardware ports.

Type and integer shape parameters resolve before constant/function parameters. The compiler retains a typed binding record containing the canonical constant value or exact callable identity, dependency closure, evaluator schema, and content digest. Source spelling is absent from semantic, artifact, and cache identity. Defaults for constant/function parameters are unsupported.

3.4 Compile-time integer modulo

Use `%` with integer constants, specialized value parameters and generated indices. It is accepted in generic value arguments, static indices and compile-time `if` conditions, not runtime hardware expressions. The compile-time integer chapter owns the floor-modulo semantics, negative and zero-divisor cases, bounds and runnable PSS example.

3.5 Compile-time functions and selection

Compile-time evaluation is bounded and deterministic. Structural intrinsics are built in rather than stdlib overloads:

- `length(vector)`;
- `floor_log2(n)`, `ceil_log2(n)`, `index_width(n)`, `is_power_of_two(n)`;
- `pi()`, `sin(x)`, `cos(x)`, `log2(x)`, `log(base,x)`, `exp(x)`, and `sqrt(x)`.

Real intrinsics use a versioned decimal evaluator rather than host binary floating point. A non-integral real enters hardware only through explicit fixed-point quantization. `1 / sqrt(x)` is the compile-time reciprocal square root form used when generating seed or approximation tables.

`index_width(n)` is the storage width for an index into `n` elements: `max(1, ceil_log2(n))`. Its argument must be a positive compile-time integer expression. Integer parameter defaults are resolved in declaration order and may reference only earlier parameters:

```
module Table<N=8,IW=index_width(N)> {
    // Ports and implementation omitted.
}
```

A forward reference such as `module Bad<IW=index_width(N),N=8>` is rejected.

Specialization-time `if` accepts value parameters and canonical type parameters:

```
fn type_flag<type T>(x : T) {
    if T == u8 { 1 } else { 0 }
}

module Stage<D=1> {
    out y : u8
    if D == 1 { y = 1 } else { y = 2 }
}
```

Only the selected branch is semantically checked. Runtime signals are rejected in compile-time conditions with a suggestion to use `?:`, `mux`, `switch`, or a guarded action.

3.6 Functional datapath

Ranges are half-open and compile-time bounded:

```
products = generate(i in 0..8) a[i] * b[i]
doubled  = map(i in 0..8) { a[i] + a[i] }
y        = reduce(+, products)
z        = sum(i in 0..8) a[i] * b[i]
d        = dot(a, b)
```

`generate` and `map` produce `vec<N,T>`. `reduce` supports `+`, `*`, `&`, `|`, and `^`; `sum` is addition reduction. Reduction order is a deterministic source-order balanced tree, so finite-width intermediate types are stable across backends. `dot` uses full-width products followed by that same reduction.

For a nominal struct element, only `sum`/`reduce(+, ...)` is defined. Every internal node resolves the ordinary exact `operator +` overload for the two types produced by its children; the overload result is the exact type presented to the next node. Small and otherwise ineligible reductions retain that complete typed tree directly. Eligible large pure bounded regions instead retain a `FunctionalRegion` plus an `ExactReductionPlan`; the plan records the same balanced topology, exact intermediate result types, and resolved overload/callee identity without cloning every call body. These are two storage forms for one language contract, not two reduction semantics.

There is no component-wise field rule, implicit conversion, identity insertion, or special case for `Complex`. The frozen reduction tree is not reassociated, and explicit quantization remains exactly where the source or overload body placed it.

Range bounds may use resolved module value parameters. Runtime ranges, empty value-producing ranges, and unbounded generation are rejected.

Compile-time iterator arithmetic is mathematical integer arithmetic. It does not require hardware `extend`/`truncate` noise:

```
reversed = generate(i in 0..48) bits[47 - i]
```

For a vector, `values[first..past_last]` is a half-open compile-time range and lowers to the same ordered static element references as the corresponding vector literal. It is not a runtime slice and does not change packed bit-slice syntax `raw[MSB:LSB]`.

A functional iterator is a compile-time integer and may feed a generic value specialization directly or through a bounded constant expression:

```
fn coefficient<K>() -> u8 { K }

values = generate(k in 0..4) coefficient<K=k>()
next   = map(k in 0..4) { coefficient<K=k+1>() }
total  = sum(k in 0..4) coefficient<K=k>()
```

The evaluated value, not the binder spelling, participates in specialization identity. Runtime values are not specialization constants, and an iterator binder cannot shadow a module parameter.

3.6.1 Scalable bounded functional elaboration

Large compile-time `generate`/`map` regions with a binder-invariant result type may use the compact functional representation described above. The compiler still type-checks the region with the ordinary exact rules and counts its logical elements against the same bounded generation budget (currently at most 4096); it does not defer type checking to a backend or create runtime loops. Callable specializations remain ordinary monomorphic definitions, and each use retains its own source origin.

The accepted whole-vector IFFT64 reference exercises an outer 64-lane `generate`, 64 exact products per lane, nominal complex reduction, and one final quantization. N=64 is validated through semantic analysis, canonical round-trip, and the simulator; N=8/N=16 exercise Direct-SV and Verilator. This evidence does not claim full 4096-multiplier N=64 RTL or a production streaming architecture. See the 802.11a validation report for the measured boundary and the later streaming implementation.

A fully compile-time generated vector may initialize an immutable ROM without external source generation:

```
fn coefficients<N>() {
    generate(i in 0..N) truncate<8>(i * i)
}

rom table : rom<u8,N> {
    read_latency 1
    init coefficients<N=N>()
}
```

The complete transitive function/intrinsic dependency identity participates in the ROM and companion hashes. Compile-time functions still have no filesystem, network, process, clock, state, random, or environment access; only the compiler publishes a validated image. The initializer must resolve to exactly the declared vector shape and element type.

3.7 Typed bit and collection composition

Bit slicing and composition are expressions rather than backend spellings:

```
high   : bits<4> = word[7:4]
flag   : bit = word[0]
joined : bits<8> = concat(high, word[3:0])
same   : u8 = bitcast<u8>(joined)
flat   : vec<8,u8> = reshape(matrix)
```

Slices use inclusive compile-time `[MSB:LSB]` bounds. A single packed-bit selection `raw[index]` returns `bit`, uses conventional LSB-zero numbering, and allows a range-proven runtime unsigned index. A fixed-width packed slice `raw[offset +: WIDTH]` returns `bits<WIDTH>` after the same bounds proof. Vector sequence indexing uses the same low-first convention: `vec` element zero occupies the least-significant packed element region. Scalar `concat` places its first operand at the MSB; homogeneous-vector `concat` preserves collection order and returns a vector. `reshape` changes only nested vector shape. `bitcast<T>` uses the canonical layout described in Types and numerics: struct declaration field zero is at the MSB, while vector element zero and tuple `item0` are at the LSB. It requires exact packed width and performs no resize or scale change. A source or target containing a nominal enum is rejected; enum representation is not a public bitcast contract. `pack` and `unpack<T>` remain low-level compatibility spellings.

For representation-bit checks, `reduce(^, scalar)` and `parity(scalar)` operate on the stored bits of `bit`, raw bits, and signed/unsigned integers. Reducing a general vector retains the existing element-wise reduction semantics.

3.8 Generic structs, functions, and operators

Generic parameters have explicit kinds:

```
struct Pair<type T> { left:T right:T }

fn duplicate<type T>(x:T) { x + x }

module Use {
    in x : u8
    out y : u9
    y = duplicate(x)
}
```

Calls support exact call-site inference or explicit specialization. Generic struct constructors infer their specialization from fields where unambiguous. Every accepted call is monomorphized; there is no runtime dispatch.

A direct integer literal may use a concrete formal argument type after explicit generic arguments or non-literal arguments have fixed that type:

```
first  = same<T=u16>(1)
second = combine(1, existing_u16)
third  = combine(existing_u16, 1)
```

If a type is inferable only from literals, each literal first receives its own minimum-width exact type and normal exact unification applies. Argument order does not select a wider type. Compound expressions never receive this literal-only contextual treatment.

Libraries may overload binary `+`, `-`, `*` and unary `-` for nominal structs. Built-in scalar/fixed operations cannot be replaced. Resolution uses exact type unification, prefers an exact concrete overload over a generic one, performs no implicit conversion, and enforces nominal-owner coherence. Specialized bodies are retained once as deterministic monomorphic callable definitions. Each use site is a typed `Call` carrying the exact callee identity, signature, result type, and its own source origin. Analyses that require concrete arithmetic use the shared bounded call-expansion service; nominal reductions remain opaque to exact reduction rules. Direct-SV emits one `function automatic` per specialization rather than cloning the body at every call site.

`std.math.complex` is ordinary ZLang source defining `Complex<T>`, arithmetic operators, `Butterfly<S,D>`, and explicit component quantization. For example:

```
Complex<fixed<18,16>> * Complex<fixed<16,14>>
    -> Complex<fixed<35,30>>
```

The scale-30 result cannot be added to a scale-16 value until explicitly quantized. Its `complex_sum(values)` helper is a discoverability alias that returns the same typed `Reduce` as `sum(values)`; it adds no compiler behavior. Generics and nominal reductions do not imply rescaling, rounding, saturation, or casts.

Traits, runtime polymorphism, generic methods, function-name overloading, higher-order functions, recursion extensions, and implicit conversions are unsupported.

4 Compile-time integer modulo

`%` is available for integer expressions resolved during elaboration: generic value arguments, static indices, compile-time `if` conditions, and expressions whose operands are constants or specialized integer parameters. It has the same precedence as `*` and `/`, with left-associative product chains.

The quick reference summarizes the operator's scope. This example selects a chip from a constant PSS sequence; each specialization produces a constant `s2`:

```
fn nr_pss_chip<ID,K>() -> s2 {
    mseq = 0x42afa51b8fe1de592044c5d6c19a9cf6
    mseq[(K + 43 * ID) % 127] ? -1 : 1
}

module PssChip {
    out chip : s2 = nr_pss_chip<ID=2,K=126>()
}
```

Modulo uses floor-division semantics: a nonzero result has the sign of the divisor. For example, `-5 % 3 == 1`, `5 % -3 == -1`, and `-5 % -3 == -2`. With a positive divisor `D`, the result `r` satisfies `0 <= r < D`, including for a negative dividend. A zero divisor is an error in an evaluated expression; unselected compile-time `if` branches are not evaluated.

Generic value arguments can themselves contain `%`, such as `nr_pss_chip<ID=2,K=(126 + 1) % 127>()`. A condition such as `if K % 2 == 0` selects a branch during specialization, not a runtime circuit. Static index results still undergo the normal bounds checks; `%` does not enable unchecked indexing or change hardware arithmetic widths.

Both operands must be compile-time integers. Input ports, registers and ordinary function arguments are runtime values, even if a caller supplies a constant or a register has a reset initializer. Runtime remainder is not a hardware operator, and this feature does not infer a divider or a fixed-point remainder. Bounded generated specializations fold modulo with concrete loop indices; they remain subject to the existing elaboration limits.

5 Sequential state, rules, pipelines, and storage

ZLang keeps combinational values, next-state actions, timing, storage, and physical clock ownership explicit in source. Synchronous active-high reset is the default. Each independently declared clock has one reset contract and may own ordinary state in the same module.

5.1 Clock and reset

Single-domain modules declare one pair:

```
clock clk
reset rst
```

Because newlines are ordinary whitespace, the compact adjacent spelling is equivalent and introduces no combined declaration or inferred domain:

```
clock clk reset rst
```

Clock and reset are declared together. Multiple domains require explicit reset, port, and state ownership wherever inference would be ambiguous:

```
clock source_clock
reset source_reset @source_clock

clock destination_clock
reset destination_reset @destination_clock

reg source_count : u32 @source_clock = 0
reg destination_count : u32 @destination_clock = 0

rule TickSource @source_clock when 1 {
    source_count <- truncate<32>(source_count + 1)
}

rule TickDestination @destination_clock when 1 {
    destination_count <- truncate<32>(destination_count + 2)
}
```

An unannotated stateful declaration remains concise in a single-clock module. In a multi-clock module its domain may be inferred from one unique destination or dynamic operand domain; otherwise analysis reports `ZL-DOMAIN-AMBIGUOUS`. Constants are domain-neutral. Combining dynamic values from different domains, including through an unannotated local, reports `ZL-DOMAIN-CROSSING`.

Rules and priorities are scheduled within a domain. Rules in unrelated domains do not acquire cross-clock mutual exclusion or atomicity. FSM-generated state and rules inherit the FSM annotation. Exact `pipeline(N) @clk { ... }`, FIFO, memory, ROM, and CSR state similarly retain one resolved owner; inserted pipeline and balancing registers never act as CDC.

Implicit clock-domain crossing is always rejected. See Hierarchy and protocols.

The recommended asynchronous spelling asserts immediately and releases only after two active clock edges:

```
clock clk
async reset arst_n @clk { polarity active_low }
```

Normal state transition resumes on the third edge after external deassertion. Use the externally synchronized release contract only when the external source already guarantees synchronous deassertion to this exact clock. It removes the internal 2FF while retaining asynchronous reset for state with a declared reset value. The low-level `reset ... { mode asynchronous ... }` spelling retains raw native deassertion for compatibility. Exact polarity, falling-edge behavior, hierarchical propagation, and unsupported formal/target routes are documented in the physical clock/reset contract.

5.2 Registers and next state

```
reg count : u8 = 0
count <- truncate<8>(count + 1)
```

The explicit multi-clock spelling places the annotation before the initializer:

```
reg count : u8 @datapath_clk = 0
```

Expressions read committed beginning-of-cycle state. All accepted `<-` updates become visible together at the active edge. A register without an accepted update holds. Reset restores the declared constant. Multiple unordered writers are rejected.

The reset value is optional. `reg retained : u8` has no generated reset branch or reset sensitivity; its hardware value is unspecified until written. Native two-state simulation starts it from a

deterministic zero seed for tool parity, which is not a power-up or reset guarantee in generated RTL.

Use `zlang sim --logic-state` to opt into the per-bit simulation lattice `0`/`1`/`U`/`X`. State without an initializer begins `U` (never refreshed). An accepted state write converts unresolved `U` bits to `X` (refreshed but unknown), while a non-write holds `U`. Exact controlling values still resolve: `U | 1` and `X | 1` are `1`, while `U & 0` and `X & 0` are `0`. `--strict-uninitialized` implies logic-state execution and turns a typed read of either `U` or `X` into an error that identifies which state was observed.

Observations belong to the selected implementation. Exact binary optimization need not preserve the original expression's U/X propagation, and strict mode does not diagnose every cancelled operand in the source. U/X tags are not an algebraic hardware domain; unknown stateful guards or memory addresses still fail rather than choosing a deterministic branch or location.

Python callers may use `set_logic`, `get_logic`, `outputs_logic`, and `Program.create(initial_registers=...)`. Packed spellings are MSB-first, case-insensitive `0`, `1`, `u`, or `x`; a single character broadcasts to the full width. Register names use the design's state paths, including explicit child paths such as `child.counter`; ambiguous or non-register names fail before any state is changed. An edge that fails because of an unknown stateful control or native execution error rolls back logic state, event flags, and trace state atomically. CLI equivalents are `--set-logic` and `--initial-reg`. Standard FST/VCD represent both `U` and `X` as `x`; ZLang emits a same-width hidden companion `__zlang_meta/*_u_mask` whose set bits distinguish `U`. `--trace-signal` may be repeated to select the traced surface, and the initial sample is emitted at time zero. The waveform is an event-index visualization: each JSONL event occupies two event-domain trace ticks, reset changes precede that event's selected clock edges, and falling-edge, active-low, and coincident-domain levels follow the compiled domain contract. It does not reconstruct analog time or simulator delta-cycle regions that are absent from the event schedule.

This is not full Verilog simulation: high impedance (`Z`), electrical resolution/strength, and Verilog event-region semantics are not modeled. Ordinary simulation remains deterministic two-state simulation, and generated Direct-SV is unchanged by these simulator options.

An ordinary output is a combinational wire. Output-visible state is declared explicitly with `out reg`:

```
out remaining : s32              // combinational wire
out reg accepted : s32 = 0       // resettable stored output
out reg retained : u8 @data_clk  // stored output with no reset branch
```

A registered output is one state resource and the public output port at the same time. It is readable inside the module, accepts `<-` updates, and holds when no accepted action updates it. Its optional initializer is its reset value; omitting it has the same unspecified-hardware-reset semantics as an ordinary register without an initializer. The type must be register-storable. Ready/valid, credit, packet, request/response and other protocol endpoints are not `out reg` ports; their individual protocol fields retain their existing ownership and flow-control rules.

ZLang `out reg` names actual architectural storage. It is not the historical Verilog/SystemVerilog `output reg` declaration category: a ZLang registered output has hold behavior, one explicit state owner, and the same clock/reset and atomic-update rules as an ordinary ZLang register.

Control registers may use nominal enums. Reset and next-state values must be members of the exact same declaration, and an exhaustive enum `switch` is the preferred finite-state-machine spelling:

```
enum Phase { Idle Active Done }
reg phase : Phase = Phase.Idle
phase <- switch phase {
    Phase.Idle => start ? Phase.Active : Phase.Idle
    Phase.Active => finish ? Phase.Done : Phase.Active
    Phase.Done => Phase.Idle
}
```

The semantic simulator and Direct-SV share the declaration-order ordinal encoding. The source-

authored APB bridge in `stdlib/bus/apb.zhl` uses an enum for its internal phase without exposing it through the external bus ABI.

For control-oriented state, `fsm` is concise syntax for the same enum register and guarded atomic rules:

```
enum TxPhase { Idle Header Payload Done }

out reg remaining : u32 = 0

fsm phase = TxPhase.Idle {
    Idle {
        when start -> Header { remaining <- length }
    }
    Header { when header_sent -> Payload {} }
    Payload {
        priority {
            when abort -> Idle {}
            when last -> Done {}
        }
    }
    Done { -> Idle {} }
}
```

The qualified initial member in `fsm phase = TxPhase.Idle` infers the exact nominal enum type. The explicit compatibility spelling `fsm phase : TxPhase = Idle` remains accepted; neither form infers from guards, actions, or transition targets.

Every member appears exactly once. A state body is `hold`, one transition, or an explicit `priority` block; source order never creates hidden priority. The state update and transition actions form one ordinary atomic rule, reset restores the declared initial member, and direct writes to the generated state register are rejected. `fsm` is lowered before canonical/backend IR and does not add a second scheduler or runtime procedural `if`.

The production Wi-Fi `IeeeIFFTInputStrip` is a second source-authored witness: its `Data`/`Flush` FSM atomically starts and retires a 64-transfer zero-frame flush, holds under backpressure, and returns to `Data` after the last accepted sample. Mid-flush reset restores the declared initial state and discards the incomplete protocol epoch.

5.2.1 Runtime-indexed vector register updates

A one-dimensional vector register supports one range-proven runtime element write in an action group:

```
reg samples : vec<64,u8> = generate(i in 0..64) 0

when capture {
    samples[index] <- value
}
```

This is exactly `samples <- VectorUpdate(samples,index,value)` over the beginning-of-cycle snapshot. The unsigned index must be statically proven in range. A second dynamic or static write to the same vector register conflicts; nested paths, slices, runtime-selected instances, and automatic memory inference remain outside this bounded form.

A rule guard may establish the required bound through a simple unsigned comparison and conjunction:

```
when (index < 64) & capture {
    samples[index] <- value
}
```

That fact is scoped to this rule only. The bounded refinement understands these simple comparisons/conjunctions; `index <= 64`, disjunctions, and uses outside the guarded rule do not prove the required `0..63` range.

5.3 Delays and fixed pipelines

```
delayed = delay<2>(value)
piped = pipeline(3) { expression }
```

`delay<N>` adds exactly `N` zero-reset scalar stages. `pipeline(N)` is an exact visible-latency contract: for a supported pure scalar expression DAG, planning may place real computation boundaries within those `N` cycles and inserts the required reconvergent-path balancing. It never changes the requested latency or II, and `pipeline(1)` remains exactly one cycle. Unsupported nodes fail closed; the emitter does not fall back to an unreported whole-expression output delay. Outside a scheduled pipeline region, non-timeless operands at an operator must still be latency-aligned explicitly.

Compiler-selected latency/resource alternatives use `implement { expression intent { ... } }`. Scalar `pipeline(auto)` is retired; protocol `transform pipeline(auto, ...)` is a separate ready/valid construct.

5.4 Exact module timing contracts

A scalar datapath module may publish one exact contract shared by all of its wire outputs:

```
module TimedChild {
    clock clk
    reset rst
    in x : u8
    out y : u8

    timing {
        latency 4
        ii 1
    }

    y = pipeline(4) { x }
}
```

The contract describes observable behavior; it does not itself insert registers or authorize retiming. Constants are timeless, ordinary input-dependent logic is known at latency zero, and explicit `delay<N>`/`pipeline(N)` adds exactly `N`. A supported pipeline region may use its selected schedule to place and balance those internal cuts. Timeless values may join a known path; joins outside that graph still require equal known latency.

Registers, rules, FIFO/memory/ROM observations, protocols, and uncontracted child outputs are `unknown` under the current bounded contract. A positive latency requires exactly one clock/reset domain. Only `ii 1` is accepted. A contracted child adds its declared latency to the common known latency of its bound scalar inputs; an unaligned parent join is rejected instead of silently balanced.

This is distinct from `implement` constraints, target estimates, and measured evidence. A selected profile must preserve an exact module contract as immutable public behavior.

5.5 Guarded atomic actions

The concise action form is:

```
when increment {
    count <- truncate<8>(count + 1)
}
```

`<-` is exclusively a state update. It may target an ordinary register, an `out reg`, or a supported indexed vector-register element. An ordinary `out` wire is never stored and therefore cannot be updated with `<-`. A transient rule/FSM output uses `drive` instead:

```
out fired : bit

rule publish when accept {
    drive fired = 1
}
```

The driven wire is zero when no owning action fires. It participates in the same atomic scheduling, conflicts and explicit priority as the rule's other effects. `drive` cannot target a register or `out reg`, and a registered output cannot also have a combinational assignment. Because changing a wire update to stored state is a semantic choice, the compiler reports both alternatives but does not apply an automatic editor fix.

`drive` is a logical transient output assignment. It does not model a tri-state driver, high impedance (`Z`), electrical or pad drive strength, analog behavior, or Verilog multiple-driver resolution. Its zero default is therefore pulse/event semantics, not retained state.

Form	Cycle semantics	What it is not
`out value : T with value = expression`	Continuously defined combinational value	Stored state
`out reg value : T [@clock] [= reset] with value <- expression`	Clocked architectural state that holds when not updated	Verilog's historical output declaration category
`drive value = expression` in an action	Scheduled transient wire effect; zero when no owning action fires	Tri-state/electrical drive or implicit storage

Named compatibility spelling remains supported:

```
rule increment_count when increment {
    count <- truncate<8>(count + 1)
}
```

Action blocks may select effects recursively with runtime `when`, `else when`, and `else`:

```
fault_update: when fault {
    when valid_state {
        overflow_state <- 1
    } else {
        valid_state <- 1
        overflow_state <- 0
        sticky_state <- fault_code
    }
} else when clear_valid {
    valid_state <- 0
    overflow_state <- 0
} else when clear_overflow {
    overflow_state <- 0
}
```

An `else` binds to the nearest unmatched `when`, and an `else when` chain selects its first true predicate. A false `when` without an `else` contributes no effects; later statements in the enclosing action block are still considered, so multiple sibling conditionals can contribute to the same transaction.

Read this as two operations that never feed back into each other: predicates first choose the effects, then the scheduler either commits the complete chosen transaction or commits nothing.

Storage readiness and rule conflicts belong only to the second operation; they can suppress a transaction but can never make an `else` branch run.

Every guard and operand reads the same pre-edge snapshot. The complete tree retains one outer `Rule`, one rule-fire identity, and one `ActionGroup`; selected effects and unconditional surrounding effects commit together or not at all. If no effect is active, the rule does not fire. Source order creates neither state visibility nor implicit priority, and reset suppresses the complete group.

Branch choice depends only on the predicates. If the selected branch contains an illegal FIFO or memory action, the whole action group is suppressed; it does not fall back to an `else` branch because another branch's storage action would be ready. The scheduler checks conflicts and storage legality using only active effects. Explicit `drive` output writes are scheduling resources too: a rule that loses a priority conflict on an output cannot commit its otherwise unrelated state effects.

Each runtime action condition must have exact type `bit`. Opposite structured branches may write the same resource, but writes on potentially overlapping paths are rejected; the compiler does not attempt arbitrary Boolean theorem proving. An explicitly empty branch is a valid no-op, while an entire action tree with no possible effect is rejected.

Runtime action selection is distinct from both other selection forms:

- compile-time `if` chooses declarations or values during elaboration and is not an action statement;
- `condition ? a : b`, `mux`, and `switch` select pure combinational values;
- runtime `when` selects effects that participate in one cycle-level atomic transaction.

Rules read beginning-of-cycle state. If the guard is true, all resource actions are legal, and scheduling permits the rule, its actions commit atomically. Reset suppresses all rules. Conflicting writers require explicit priority:

```
priority {
    clear: when clear_request { count <- 0 }
    increment: when increment_request {
        count <- truncate<8>(count + 1)
    }
}
```

Priority suppresses a lower enabled rule only when it conflicts with a fired higher rule. It is not procedural `if`/`else`: nonconflicting enabled rules may fire together.

A simple ordered chain may instead be written as:

```
priority first > second > third
```

This syntax expands to the adjacent edges `first > second` and `second > third`. Only declared edges participate; source order adds no priority. Use the block/pair form for any other priority graph.

5.6 FIFOs

```
fifo queue : fifo<u8,4>
queue.data = rx.payload
queue.push = rx.transfer
queue.pop = tx.transfer
```

Read-only observations include `.front`, `.count`, `.full`, `.empty`, `.ready`, `.valid`, `.overflow`, and `.underflow`. A legal simultaneous pop/push while full preserves occupancy and ordering. An empty pop cannot consume a same-cycle push.

FIFO depth may be a positive compile-time value expression. Backends receive the resolved integer, not a hardware parameter.

Rules can instead own FIFO operations atomically:

```
rule replace when rotate {
    queue.pop()
    queue.push(queue.front)
    accepted <- 1
}
```

Rule-owned FIFO actions and globally driven `.push`/`.pop` controls are mutually exclusive ownership forms. An illegal resource action suppresses the whole rule.

Rule scheduling selects the lexicographically highest-priority **legal atomic set**, not each rule independently. Thus a push on a full FIFO may fire with a compatible lower-priority pop on the same edge; a push alone remains blocked. The compiler constructs exact selection regions for empty, interior, and full occupancy without enumerating every count or rule subset. If exact scheduling exceeds its bounded work budget, compilation reports `ZL-STATE-SCHEDULER-LIMIT` rather than emitting an approximate circuit.

5.7 Writable memories

One-read/one-write memories may optionally expose a byte write mask. The existing one-cycle, reset-cleared form remains the default:

```
in write_mask : bits<4>
memory table : mem<u32,256> {
    read_latency 1
    collision write_first
}
table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
table.write_mask = write_mask
```

The mask type is exactly `bits<ceil(element_width / 8)>`. `write_mask[0]` controls the least-significant byte. For a width that is not a multiple of eight, the final mask bit controls only the remaining live most-significant bits; padding bits are never stored. Disabled lanes retain their old bits. For a same-address `write_first` access, the registered read result is the post-mask merged word, not the unmasked input. Omitting `write_mask` preserves the existing full-word write behavior. Consequently, unmasked and masked memory elements may have any positive recursively bit-packable, non-enum width.

Rule-owned memories accept the same operation as an optional third operand:

```
rule store when enable { table.write(address, data, byte_mask) }
```

Within a masked rule-owned memory, the compatible two-operand form `table.write(address, data)` means an all-lanes write. The mask does not add a read enable, another port, a new clock domain, or target-specific BRAM mapping.

`read_latency` counts active edges in the memory's read domain between a captured address and the visible result. These are compiler/simulator/Direct-SV contracts, not promises that a particular FPGA RAM primitive will be inferred:

Storage form	Accepted `read_latency`	Tested behavior and physical boundary
Globally controlled `mem`, including named same-clock ports	`0..16`	`0` is combinational; `1` is registered; `2..16` add exact read-domain register stages. Cycle tests cover `0`, `1`, `2`, and the upper bound `16`; Direct-SV is linted.

Storage form	Accepted `read_latency`	Tested behavior and physical boundary
Rule-owned legacy `mem`	Exactly `1`	A selected read action captures at an edge and holds its result otherwise; rule/storage simulations and Direct-SV tests cover it.
`async_mem` 1W1R	`1..16`	Reader-domain registered output; cycle tests cover `1`, `2`, `3`, and `16`, including unrelated clocks. Generic Direct-SV supports `old`; stronger cross-clock collision claims need exact target evidence.
Immutable `rom`	Exactly `1`	Synchronous table read; compile, simulation, and Direct-SV tests cover it.
Internal memory of `async_fifo`	Fixed `1`	The FIFO prefetch controller accounts for the registered read; this is not a user-settable FIFO latency.

The current explicit Xilinx 7-Series synchronous-memory physical route accepts only `read_latency 1` and matching reset, mask, port, and collision capability. Other legal latencies can use generic RTL under a preferred policy; a required native route rejects them. `read_latency 0` never claims synchronous BRAM.

5.7.1 Named same-clock ports

Leaving the body without port declarations preserves the legacy controls and their generated behavior. A memory may instead declare up to eight named logical ports:

```
memory table : mem<u32,1024> @clk {
    read_write_port a
    read_write_port b
    init RESET_WORD
    read_latency 1
    collision old
    write_priority a > b
}

table.a.read_enable = a_read
table.a.write_enable = a_write
table.a.address = a_address
table.a.write_data = a_data
a_data_out = table.a.read_data
```

`read_port`, `write_port`, and `read_write_port` state the exact access shape. All named ports of an ordinary `mem` have one resolved clock domain. When more than one port can write, `write_priority` must list every writer exactly once. Priority affects only writes to the same address; simultaneous writes to different addresses both commit. `collision old`, `new`, and `no_change` select the same-clock registered-read result. The older spellings `read_first` and `write_first` remain accepted aliases for `old` and `new`.

Named-port masks retain the arbitrary-bitwidth lane rule above. Mixed port widths and asymmetric depths are rejected. Rule-local memory actions remain a property of the legacy implicit 1R1W form and cannot be mixed with named ports.

An optional `init VALUE` supplies one exact compile-time element value for every cell. It may be a typed module value parameter. With `contents clear`, generic/register-array hardware writes

that value to every cell on reset. With `contents preserve`, it is only the deterministic power-up value; FPGA block RAM maps it to bitstream initialization and does not pretend that a run-time reset rewrites the array. Omitting `init` retains the exact zero value used by the existing reset/initialization policies. Full per-address images remain the role of immutable `rom` in the current language contract.

When a memory exactly matches a target resource, that resource is used. A `1W+nR` shape otherwise becomes coherent replicated 1R1W storage; other same-clock multiwrite shapes may use a deterministic register array, read muxes, and priority gates when the logical storage is at most 4096 bits. Larger unsupported shapes fail with a diagnostic recommending explicit banking, arbitration, or standard-library composition. No automatic banking is performed because arbitrary same-bank collisions need a protocol contract.

5.7.2 Explicit dual-clock memory

`async_mem<T,N>` is a separate, deliberately narrow semantic resource:

```
memory table : async_mem<u32,1024> {
    write_port wr @write_clk
    read_port rd @read_clk
    init INITIAL_WORD
    read_latency 1
    collision old
    reset { contents preserve read_data clear }
}

table.wr.enable = write_enable
table.wr.address = write_address
table.wr.data = write_data
table.wr.mask = write_mask
table.rd.address = read_address
read_data = table.rd.data
```

It has exactly one write-only port and one read-only port in different explicit domains. Cells change only on the writer edge; `rd.data` changes only on the reader edge and has destination-domain provenance. The contents reset policy belongs to the write domain and the read-result policy to the read domain. Direct use of foreign-domain controls is rejected; a pipeline register is not a CDC primitive.

`async_mem` requires `read_latency` in `1..16`. The generic implementation captures the cell on the reader edge and, for latency greater than one, shifts the captured value through reader-domain output registers. This adds visible latency without changing the contents or crossing domains. `read_latency 0` is invalid for `async_mem`; it would be an unregistered cross-clock read, not a block-RAM read port.

For coincident logical edges the simulator uses a pre-edge snapshot: `old` returns the prior cell, while `new` forwards the coincident write. This is a precise digital model, not an analog metastability or silicon timing guarantee. Portable generic Direct-SV publishes `old`; stricter or other collision claims require an exact target capability. Asynchronous 2RW, mixed-width ports, ECC, and dual-clock ROM remain deferred.

```
memory table : mem<u8,16> {
    read_latency 1
    collision read_first
}

table.read_address = read_address
table.write_enable = write_enable
table.write_address = write_address
table.write_data = write_data
read_data = table.read_data
```

Alternatively, one memory can be owned by atomic rules:

```
rule fetch when read_enable {
    table.read(address)
    accepted <- 1
}

rule store when write_enable {
    table.write(address, write_data)
}
```

`read` registers `table.read_data` one cycle later and it holds when no read fires. `write` commits at the selected edge. One selected read and one selected write may fire together; same-address behavior follows the declared collision mode. Their operands and all other rule effects observe the same pre-edge snapshot. Reset suppresses actions. With the default profile it clears both cells and `read_data`.

Global controls and rule actions cannot be mixed. The bounded scheduled form supports one memory per module, alongside registers and scheduled FIFOs; two reads or two writes conflict and require existing explicit rule priority.

Executable memory elements may be recursively bit-packable non-enum aggregates, as well as scalars; depth is a power of two and at least two. A globally controlled ordinary `mem` accepts `read_latency` in `0..16`: zero is a combinational read; each positive value is an exact number of domain-local read edges from address capture to the visible result. Rule-owned memories remain exactly one-cycle because their read action is selected at an edge. Zero-cycle ordinary `mem` is a generic combinational storage behavior, not a native synchronous BRAM claim. Native target binding requires an explicitly matching clocked-read capability. Until a target mapping proves its internal output-register configuration and any additional fabric stages, a requested latency greater than the advertised native latency uses generic storage under a preferred policy or fails under a required policy.

Cell and visible read-result reset behavior may be selected independently:

```
memory table : mem<u32,256> {
    read_latency 0
    collision read_first
    reset {
        contents preserve
        read_data preserve
    }
}
```

If the `reset` block is present, both directives are required. If it is absent, the exact default is `contents clear` and `read_data clear`. Reset always suppresses writes and scheduled actions. At latency zero, `read_data clear` masks the combinational result while reset is asserted; `read_data preserve` leaves the addressed preserved cell visible. At latency one, the corresponding policy clears or holds the result register. `read_first` observes the old cell before a coincident write edge; `write_first` observes the fully byte-mask-merged write word.

The simulator and generic Direct-SV initialize executable preserved memory state to its declared uniform `init` word, or zero when omitted, but runtime reset does not recreate that initialization under `contents preserve`. The Xilinx same-clock true-dual mapping therefore requires preserved contents; Vivado maps its source initializer to BRAM initialization. Target selection stays fail-closed when a resource does not advertise initialization support. Full/partial writable-memory images, automatic banking, unbounded multiport memory, and asynchronous 2RW are not part of this hardware surface. Bounded named-port selection is described above. The writable-memory contract defines the hardware boundary.

5.7.3 Replicated read ports, wrappers, and banking

Named ports now provide a bounded native logical model; source composition is still preferable when banking or arbitration policy is architecturally significant. The validated `ZtpuBankedMemory` uses four banks and two read replicas per bank: both replicas receive the same decoded, byte-masked synchronous write, while each read address selects its own replica and bank. This produces eight distinct physical 1R1W memories, one shared leaf specialization, two combinational read results, and one logical write port.

The concrete witness contains 256 32-bit words, split into four banks of 64 words. Its runtime reset suppresses writes and preserves both cells and combinational read data; executable simulation begins from deterministic zero contents. The semantic simulator and Direct-SV agree on full and per-byte writes, independent reads, same-address `read_first` behavior, reset, and replica coherence. This is source composition, not a new memory semantic or backend name-based rewrite. The ordinary stdlib modules `StorageDualPortMemory`, `Storage2R1WMemory`, and `StorageAsyncMemory1W1R` expose common shapes while lowering through the same memory IR. `StorageAsyncFifo` similarly wraps the existing ready/valid `async_fifo(D)` crossing; a normal `fifo<T,N>` never becomes asynchronous.

The FIFO crossing now decomposes into an internal typed `async_mem` with independent write/read clocks, `read_latency 1`, and a one-slot prefetch controller. It is not a second public memory declaration in the module. The selected AMD 7-Series `Xilinx7AsyncFifoRAMB18` or `Xilinx7AsyncFifoRAMB36` architecture binds only this internal FIFO memory to a matching independent-clock 1W1R RAM with `DO_REG=0`; unsupported width/depth/configuration fails under `required` policy and remains generic RTL under `preferred`. This route does not imply that arbitrary public `async_mem` has a native cross-clock collision guarantee.

Target-memory closure remains separate. In particular, the existing promoted OpenRAM/target macro contract does not yet match this zero-latency, preserve-on-reset profile, so the witness makes no BRAM/SRAM inference, QoR, or physical-macro claim. A byte-addressed external wrapper and target-specific latency/collision adaptation are not currently supported by this mapping.

5.8 Initialized synchronous ROMs

An immutable lookup table is declared separately from writable memory:

```
rom twiddles : rom<Complex<Twiddle>, N / 2> {
    read_latency 1
    init fft_twiddles<T=Twiddle,N=N>()
}

twiddles.read_address = phase
selected_twiddle = twiddles.read_data
```

The initializer must specialize at compile time to exactly `vec<depth,element_type>`. Elements may be bit-packable scalar/fixed values, vectors, or concrete structs; protocol/state values and nominal enums are rejected. Depth is positive and may be one or non-power-of-two. The address is an unsigned `uint<max(1,ceil_log2(depth))>` value whose proven range remains inside the declared depth.

The read is a **one-cycle synchronous read**. During reset the registered read result is zero. Reset never changes the immutable contents; after reset, the first non-reset result corresponds to the preceding accepted non-reset address. There is no hidden second output register.

Generated SystemVerilog uses one companion image. It contains one exact-width binary word per line, address zero first. Struct declaration field zero occupies the most-significant bits; vector element zero and tuple item zero occupy the least-significant component bits recursively. Fixed-point values retain their raw signed or unsigned bit pattern. Direct-SV `$readmemb` consumes that image. The CLI publishes companions beside the selected output, and artifact metadata retains the initialization dependency, evaluator, content, and file hashes. Missing or colliding companion files fail closed.

Writable or partial initialization, reloadable/asynchronous/multiport ROM, enum elements, and automatic BRAM/storage exploration remain deferred.

5.9 Stateful hierarchy

Single-domain scalar child modules may contain registers and rules. A compile-time indexed instance array may contain bounded scalar sequential or primitive ready/valid children with one matching clock/reset domain; each physical instance has independent state and reset. Aggregate scalar outputs and mixed scalar/ready-valid ports retain their exact typed leaves. Storage-only child arrays may own one FIFO, synchronous memory, or initialized ROM. The scheduled-FIFO profile may also combine FIFO actions with ordinary register writes/rules because both use the same `ResolvedTransition`; legacy globally controlled storage plus user state remains rejected.

One outer array may contain a bounded same-domain scalar or direct-ready-valid hierarchy, including an inner scalar compile-time array. First-level in-order request/response requester/responder arrays are supported with scalar wire peers and explicit indexed connections. Nested storage/CSR/aggregate protocols, nested or out-of-order request/response, other non-RV protocols, CDC, runtime-selected instances, and cross-module atomic scheduling remain fail-closed. See Hierarchy and protocols.

6 Physical clocks and resets

ZLang's `ClockDomain` records the complete physical contract of one clock and its reset. Concise declarations retain the default reset contract:

```
clock clk
reset rst @clk
```

normalizes to rising-edge clocking, synchronous active-high reset, native release, and unspecified power-up state.

6.1 Recommended asynchronous reset

The concise asynchronous form is safe for ordinary state. In a multi-clock module each annotated domain owns an independent conditioner:

```
clock clk
async reset arst @clk
```

It means asynchronous assertion followed by deassertion synchronized through two registers. Reset remains active on the first two active clock edges after the external pin is deasserted; normal state transition resumes on the third edge. Reassertion at any point immediately restarts that release sequence.

The external pin may be active-low:

```
clock clk
async reset arst_n @clk {
    polarity active_low
}
```

If reset deassertion is already synchronized to this exact clock outside the generated module, state that physical contract explicitly:

```
clock clk
async reset arst_n @clk {
    polarity active_low
    release externally_synchronized
}
```

This retains asynchronous assertion for resettable state but emits no internal two-register release conditioner. The annotation is never inferred: every destination clock domain must receive a release synchronized to that clock. Omitting `release` retains the safe internal 2FF default.

A register participates in reset only when it declares a reset value:

```
reg control : u8 = 0 // asynchronous reset to zero
reg datapath : u8    // no reset branch or reset sensitivity
```

The unreset register continues to accept enabled clocked updates while reset is asserted. Its RTL power-up value is unspecified until the design writes it. Native two-state simulation uses a deterministic zero startup seed, which is not a source-level power-up or reset guarantee. The opt-in logic-state mode represents unrefreshed bits as `U`; strict observation rejects `U` or computed `X` rather than collapsing either to a binary value. See Registers and next state.

A falling-edge clock uses falling edges for the two release cycles as well. Polarity affects the external pin and generated RTL; the simulator still accepts a logical `True` for asserted reset.

The lower-level compatibility spelling remains available for a raw native asynchronous assertion/deassertion contract:

```
clock clk { edge falling }
reset rst_n @clk {
    mode asynchronous
    polarity active_low
    power_up unspecified
}
```

This full block does not insert synchronized release. When either low-level physical block is present, every clause in that block is required. Raw active-low protocol helpers gate transfers with the deasserted high level; they do not reinterpret the physical pin as active-high.

6.2 Backend lowering

Direct-SV emits a deterministic two-register synchronizer marked with `ASYNC_REG` for the unannotated concise form. The top-level domain owns it and routes one conditioned reset through register/rule, FIFO, memory, CSR, protocol, and child state. A hierarchy does not create one synchronizer per sibling. A clocked but completely state-free module publishes the same physical contract without emitting an unused conditioner, because no reset epoch is consumed.

Within a composed artifact, each child receives the top's conditioned net and uses it directly as its asynchronous reset. Children and nested grandchildren never add another release conditioner, so hierarchy depth does not add reset release latency. The same reusable child compiled as a public top owns its own conditioner, because that compilation has a new external reset boundary.

`release externally_synchronized` routes the external reset directly into the asynchronous event control of resettable state and emits no conditioner. Resetless registers use only the active clock event.

The semantic simulator models each input item as one active edge. It therefore models immediate assertion at the sampled boundary and the exact two-edge release hold. Assertion between active edges is additionally checked in RTL simulation.

6.3 Hierarchy inheritance

A stateful child that deliberately declares no physical clock/reset may inherit the enclosing module's sole exact clock domain. This is source ownership, not an implicit adapter: edge, assertion mode, polarity, release policy/cycles, power-up policy, and physical names become the child's typed contract and specialization identity. Reusable stateful RegBus, AHB-Lite, AXI4/AXI4-Lite, AXI burst, AXI-Stream, APB, and Wishbone cores use this mechanism; source-owned flat-pin adapters and concrete application/witness tops remain explicit physical-domain owners.

Inheritance fails closed when the parent has no unique domain. A child that declares its own domain must still match one exact parent domain; no reset conversion or CDC is inferred. The

Python compilation/session API may supply one typed `root_domain` when an inherited-domain core itself is selected as a direct test or tool top. This does not create a source-language default or bypass validation: the supplied contract participates in the exact compilation cache key and produced artifacts, while absent or ambiguous ownership remains an error.

The generated implementation artifact records the contract identity, external RTL clock/reset paths, edge, assertion mode, polarity, release policy/cycles, power-up policy, and source origin. Component and physical-instance records reference that identity.

6.4 Formal applicability

Executable formal work consumes the exact declared contract. With `power_up unspecified`, the supported combinations are described below.

Formal-only accepted `rule.fire` projections use the effective reset and polarity, not a raw active-high input assumption. Descendants consume the conditioned native reset. Rising and falling active edges, active-high and active-low pins, and synchronous or asynchronous assertion are supported. An asynchronous reset may use native or exactly two active edges of synchronized release. Applicable routes are source safety/cover, bindable recursive safety, same-cycle or fixed-latency II=1 Direct-SV semantic-reference equivalence, and formal-aware selection. Asynchronous formal execution is limited to one physical domain.

Multiple supported synchronous domains may still produce independent goal-local jobs. Asynchronous execution is deliberately single-domain; this does not define a cross-domain reset relation. Same-cycle pure candidate equivalence remains reset-independent, while fixed-latency comparison uses the declared active edge and release window.

The compiler carries the same typed clock/reset contract from the selected design through RTL bindings, formal jobs and their results. Restoration rejects missing, corrupted or mismatched contracts rather than guessing a reset mode.

Prepared-artifact, bundle, harness, result-cache, and evidence recipes therefore separate otherwise identical goals that use different edges, assertion modes, polarities, or release policies. A non-executable goal still retains its exact typed contract. If no compatible artifact exists, its physical-domain identity is absent rather than guessed.

Formal traces distinguish the two reset views. `physical_reset` is the raw external pin with its declared polarity. `trace:reset` and result `reset_state` are the normalized active-high *effective* reset used by property guards, previous-cycle history, feasibility covers, fill masks, and comparison windows. For synchronized release, effective reset remains asserted for both release edges even though the external pin is already deasserted. Failure-cycle and originating-sample attribution therefore describe the real reset epoch, not merely the raw pin level.

6.5 Boundaries

One clock domain has exactly one reset. Declaring both `reset` and `async reset` for that domain is an error. Direct-SV supports multiple independently conditioned asynchronous domains; current formal execution still skips an asynchronous goal in a multi-domain module because no cross-domain reset-epoch relation is claimed. Implicit reset crossings, reset combiners, configurable synchronizer depth, glitch filters, power-on reset, and macro-selected RTL semantics are not supported.

`power_up reset` remains typed but fails closed because no common portable synthesizable initialization mechanism is frozen. DSP48 physical reset pins, elastic or variable-latency `pipeline( auto)` equivalence, CDC/reset-refinement proofs, and target BRAM reset pins remain fail closed. So do multi-domain asynchronous formal execution, general hierarchical semantic-reference equivalence, and every route with missing or incompatible domain manifests, bindings, assumptions, or observations. formal-aware selection `available` records unavailable evidence without changing eligibility; required policies fail unless the existing exact semantic-reference equivalence route executes at the requested level. No new formal property, observation, or equivalence family is introduced.

7 Hierarchy, protocols, and explicit CDC

ZLang represents modules, endpoints, ownership, clock domains, connections, and physical instances explicitly before backend lowering. Connections are never inferred by guessing generated RTL names.

7.1 Modules and instances

Modules may have compile-time type and value parameters:

```
module Engine<type T, DEPTH=4> {
    in  value : T
    out copy  : T = value
}
```

Instantiate and bind scalar inputs explicitly or by same-name shorthand:

```
inst engine : Engine<T=u8,DEPTH=4> { value }
result = engine.copy
```

Specialization identity is distinct from physical instance identity. A single-domain parent may supply an unambiguous child clock/reset implicitly; multi-domain hierarchy requires explicit compatible domains. Child protocol endpoints use `connect`, not scalar binding syntax.

A reusable component fingerprint covers its typed hardware and the deterministic transitive closure of callables that hardware actually invokes. Functions that are merely visible through a parent/import context do not participate. Thus the same parameterless child reached through different parents is emitted once, while different value/type specializations or a different reachable helper body remain distinct and fail closed on an identity conflict.

When every intermediate child conforms to one exact named interface with one protocol input and one protocol output, an option-free path can be written as:

```
input -> decode -> execute -> output
```

This lowers to the ordinary pairwise typed edges `input -> decode.rx`, `decode.tx -> execute.rx`, and `execute.tx -> output`. Intermediate names are physical instances, not guessed ports. Arrays, unnamed or ambiguous signatures, edge options, adapters, buffering, and crossings remain explicit.

One-dimensional compile-time instance arrays are structurally unrolled:

```
inst lane[N] : Lane<W>
generate(i in 0..N) {
    lane[i].x = values[i]
}
outputs = generate(i in 0..N) lane[i].y
```

Structural generation may be nested and may contain compile-time `if`/`else`. Both are elaborated by the same bounded unroller; they do not create runtime loops or dynamic topology:

```
generate(i in 0..N) {
    generate(j in 0..N) {
        if i == j { lane[i].x = values[j] }
    }
}
```

The existing total expansion limit applies across the complete nested tree. Exceeding it reports a bounded compile-time-generation diagnostic rather than partially elaborating the design.

A runtime selector may project one exact bit-packable wire output. This is a read-only mux over the already elaborated children, never a dynamic physical instance:

```
selected = lane[select].output
```

Every `lane[i]` still exists, receives its compile-time bindings, and advances its own state independently. The selector neither routes child inputs nor gates a child clock, reset, rule, or storage action. The explicit equivalent is `outputs = generate(i in 0..N) lane[i].output` followed by `selected = outputs[select]`; both spellings retain the same physical child identities.

The bounded supported form covers:

- combinational scalar children, including exact aggregate scalar outputs;
- single-domain scalar children with registers/rules;
- primitive ready/valid children, including scalar wire ports beside the endpoint and the existing one-FIFO profile;
- legacy storage-only children owning one FIFO, synchronous memory, or initialized ROM;
- the scheduled-FIFO profile in which FIFO actions and ordinary register writes share one existing `ResolvedTransition`; and
- an outer one-dimensional array whose element contains a bounded same-domain scalar or direct-ready-valid hierarchy. Inner compile-time scalar arrays are retained as distinct physical paths rather than flattened by generated name.

The first-level request/response profile is also bounded and explicit. Each array child has exactly one `ordering in_order` endpoint plus scalar wire ports, and each connection names compile-time indexed requester and responder elements:

```
generate(i in 0..N) {
    requester[i].issue = issue[i]
    responder[i].accept = accept[i]
    connect requester[i].mem -> responder[i].mem
}
```

Every physical element retains its own outstanding ledger and reset epoch. Out-of-order matching, an additional protocol on the same child, nested or transitive request/response arrays, and unindexed endpoint references fail closed. This profile has an executable capability-registry witness and is covered by compiler, simulator, and backend validation.

Legacy globally controlled storage still cannot be combined with user registers/rules. Credit and other non-RV protocols, CSR or storage below a nested array element, CDC arrays, runtime-selected inputs or protocol endpoints, cross-module atomic scheduling, and incomplete bindings are rejected.

Concise declarations normalize to the same hierarchy IR when unambiguous:

```
frontend : AXI4LiteToRegBus<32,32>
axi : AXI4Lite<32,32>.slave @clk
axi -> frontend.axi
```

Explicit `inst` remains the escape hatch for namespace ambiguity.

7.1.1 Typed external modules

A bounded scalar-wire component may declare portable behavior with an ordinary pure ZLang model:

```
extern module VendorAdd : AddIfc { model add_model }
```

The concise source form is `extern module VendorAdd : AddIfc { model add_model }`.

The applied interface and model produce one exact semantic signature. Physical SystemVerilog is supplied separately through a hash-validated `ExternalPhysicalMapping`; inline HDL never defines ZLang semantics. The supported form has non-parameterized, clockless scalar inputs

and one scalar output in Direct-SV. State, protocols, arrays, and generic external components fail closed.

7.2 Clock and reset boundary

Legacy `clock clk` plus synchronous active-high `reset rst` works through the production Direct-SV backend. A single-domain parent may supply that exact domain to a child only when the mapping is unambiguous; this is domain inheritance, not implicit CDC. Reusable stateful standard-bus cores use this same rule; source-authored pin adapters and concrete application tops remain the external domain owners. One non-default falling-edge, asynchronous, or active-low physical contract is typed and emitted by Direct-SV. Concise `async reset` adds one root-owned two-edge release conditioner and passes the conditioned reset through the closed child ABI; the raw asynchronous compatibility form remains distinct. Instance arrays require one compatible inherited domain unless they are purely combinational. Multiple independently conditioned asynchronous domains are supported in Direct-SV. Every domain of a stateful child must match exactly one parent physical clock/reset contract; no implicit reset crossing is inserted. Multi-domain asynchronous formal execution, power-on reset, and implicit reset crossing remain fail-closed.

7.3 Public top ABI

The selected top always exposes integration-friendly typed leaves. Struct fields and structural tuple components, including those inside protocol payloads, become individually named ports; tuple paths use `item0`, `item1`, and so on. A vector leaf remains one multidimensional packed SystemVerilog array; for example `vec<8,u8>` becomes `logic [7:0][7:0] samples`. It is neither an anonymous 64-bit public bus nor eight separately named ports. Nested `vec<Struct>` values become one array per struct field, and `vec<Tuple>` values become one array per tuple component.

Direct-SV RTL uses this public `TopPhysicalABI`. The conversion follows the canonical packing order with element zero in the least-significant region: for a descending packed vector dimension, source element zero maps to physical index `0`. There is no source annotation, compiler option, or backend-specific preprocessor branch selecting another public ABI.

7.4 Ready/valid

```
in  rx : rv<u8>
out tx : rv<u8>

tx.payload = rx.payload
tx.valid = rx.valid
rx.ready = tx.ready
```

For an input endpoint, payload/valid enter the module and ready leaves it. Output ownership is reversed. `.transfer` is a read-only protocol property meaning `valid & ready`. A source stalled with `valid=1` and `ready=0` must hold payload and valid stable.

Direct composition names source then sink:

```
connect rx -> tx
connect buffered_rx -> buffered_tx { buffer 2 }
```

A ready/valid buffer is real FIFO state; it does not appear implicitly.

7.5 Credit and request/response

`credit<T,N>` provides pulse-return flow control. `.transfer` is the physical send after credit gating, `.credits` is the current bounded count, and `.return` returns one credit. Counters begin at `N`.

Adapters are explicit:

```
connect rx -> tx { adapter rv_to_credit }
connect crx -> rtx { buffer 2 adapter credit_to_rv }
```

Request/response endpoints declare ordering and outstanding capacity:

```
interface mem : request_response<Request,Response> {
    max_outstanding 2
    ordering in_order
}
```

The requester owns request payload/valid and response ready; the responder owns request ready and response payload/valid. Request and response channels expose their own read-only `.transfer` events. Directional buffers are independent:

```
connect requester.mem -> responder.mem {
    request_buffer 4
    response_buffer 2
}
```

Buffered but unaccepted requests are distinct from accepted outstanding requests. The ledger increments on responder-side request transfer and decrements on requester-side response transfer. Reset begins a new protocol epoch. `ordering out_of_order` requires a matching scalar ID field; hierarchical out-of-order composition remains outside the current bounded subset.

7.6 Aggregate protocols and the standard library

The compiler-shipped `std` namespace maps to ordinary `.zhl` files below `stdlib/`. The compiler knows generic aggregate schemas, roles, ownership, hierarchy, and domains; it does not hard-code AXI/APB transactions.

Available source-authored profiles include RegBus, AXI4-Lite, APB, AXI-Stream, Wishbone B4 Classic, bridges, and the CSR target:

```
import std.bus.reg
import std.bus.axi_lite

module AxiCsrTop {
    clock clk
    reset rst
    interface axi : AXI4Lite<32,32>.slave @clk
    inst frontend : AXI4LiteToRegBus<32,32>
    inst csr : RegBusCSRTarget<32,32>
    connect axi -> frontend.axi
    connect frontend.regbus -> csr.regbus
}
```

Aggregate pass-through connects each member according to typed ownership, including reverse ready:

```
input -> output
```

Protocol member properties such as `.transfer`, FIFO observations, and request/response channel events are part of typed semantics. They are not arbitrary user-defined fields and cannot be assigned when read-only.

7.7 Packets, virtual channels, and arbitration

`packet<T>` extends ready/valid with `.last`. Arbiters explicitly select source order, `fixed_priority` or `round_robin`, and `beat` or `packet` grant lifetime. `vc_credit<T,V,C>` maintains independent credit counts per virtual channel.

7.8 CSR blocks

CSR source describes address layout, access policy, reset, and hardware binding once. Supported policies are `rw`, `ro`, `wo`, `w1c`, `pulse`, and `reserved`. Hardware status uses `<-`, commands use `->`, and sticky events make simultaneous hardware/software priority explicit. RTL, JSON, and Markdown are derived from the same typed model.

Status and sticky sources may select exact leaves of a typed aggregate input; the compiler retains and type-checks the member expression rather than deriving a flattened name:

```
struct Perf { cycles : u32 }
struct EngineStatus { busy : bit perf : Perf }

module AggregateStatusBank {
    clock clk reset rst
    in status : EngineStatus
    csr registers @0 {
        STATUS @0 { busy bit @0 ro <- status.busy }
        CYCLES @4 { value u32 @31:0 ro <- status.perf.cycles }
    }
}
```

Aggregate members used by one CSR bank must belong to that bank's clock domain. Command bindings remain scalar output ports.

Omitted bits are implicitly reserved. A storage-independent access event may observe the same bits as their single owning field:

```
module CsrAccessEventExample {
    clock clk reset rst
    in fault_status : bits<32>
    out fault_clear, registered_clear : bits<2>
    csr registers @0 {
        FAULT @0x470 {
            value bits<32> @31:0 ro <- fault_status
            clear bits<2> @1:0 on_write -> fault_clear
            clear_later bits<2> @1:0 on_write post_accept -> registered_clear
        }
    }
}
```

The event is address-qualified by the compiler and owns no field state. Its default `active_transfer` phase is combinational during the qualified access. An explicit `post_accept` phase captures the selected event value at the accepting edge, publishes it for the following cycle, and then clears it when there is no new matching access. This phase does not change CSR readback or the canonical rule that `rdata` is meaningful only for reads. Stored, external-status, and event observations are available through typed child namespaces `state`, `status`, and `events`. JSON and Markdown maps record the selected event phase.

Repeated layouts flatten into one bank and one decoder:

```
module CsrGroupExample {
    clock clk reset rst
    csr group Window { CONTROL @0 { enable bit @0 rw = 0 } }
    csr control @0 { windows : Window[13] @0x210 stride 4 }
}
```

Group counts are bounded to 1..64 and a bank to 256 expanded registers. A logical 64-bit value may retain two independently writable 32-bit software registers:

```
module CsrSplitExample {
    clock clk reset rst
    csr registers @0 {
        INPUT_BASE @0x18 split<32> value u64 rw = 0 order low_first
    }
}
```

The same `split<32>` declaration may be part of a reusable group. Every group instance receives one logical typed projection and two independently writable 32-bit words:

```
module GroupedSplitExample {
    clock clk reset rst
    csr group Window {
        CONTROL @0 { enable bit @0 rw = 0 }
        BASE @4 split<32> value u64 rw = 0 order low_first
    }
    csr registers @0 { windows : Window[2] @0 stride 12 }
}
```

This does not imply byte strobes or an atomic two-word write. Typed command capture and registered RegBus timing are ordinary source modules: `StorageSnapshot<T,reset_value>` in `std.storage.core` and `RegisteredCSRTarget<AW,DW>` in `std.bus.reg`; the compiler has no special case for either module name.

In a multi-clock module the bank declares its owner after its base address:

```
csr control @0x4000_0000 @apb_clk { /* registers */ }
```

All blocks sharing the canonical access ABI must use that one domain. A runtime CSR value consumed by another domain still requires explicit CDC.

7.9 Clock-domain crossings

Implicit CDC is an error. Existing explicit forms are:

Crossing	Supported endpoint
`sync_level`	Slowly changing `bit` level
`pulse_toggle`	Rate-limited `bit` pulse
`handshake`	One scalar ready/valid payload
`async_fifo(N)`	Ready/valid stream through a power-of-two dual-clock FIFO

```
connect source -> destination { crossing async_fifo(4) }
```

The crossing is a semantic boundary. Its output has destination-domain provenance and may feed destination-domain combinational logic, rules, FSMs, or state normally. The compiler never rewrites, balances, or resource-packs logic through the boundary, and never chooses a crossing automatically.

```
connect enable_a -> enable_b { crossing sync_level }
rule Observe @clk_b when enable_b { seen <- 1 }
```

An aggregate with exactly one forward ready/valid member may use the same `async_fifo` crossing; its complete payload is atomic. Both domains must participate in a coordinated reset episode.

The physical Direct-SV FIFO uses one dedicated 1W1R `async_mem` with a one-cycle registered read, two-stage Gray-pointer synchronizers, and one prefetched output beat. A stalled beat holds both payload and `valid`; the read/consumed pointer advances only on transfer, so that

beat still occupies capacity. Continuous accepted beats can transfer at II=1 without an extra output bubble. The synchronizers model digital CDC behavior, not analog metastability or an MTBF guarantee.

The ordinary `std.storage.core.StorageAsyncFifo<T,D>` wrapper exposes this same crossing as a reusable module with explicit writer and reader clock/reset ports. It lowers through the existing CDC IR; its module name is not special to the compiler. A normal `fifo<T,N>` never changes into an asynchronous FIFO because its endpoints happen to use different domains.

No automatic protocol adaptation or implicit CDC is performed. The source-owned `std.bus.axi4` modules provide bounded IDs, bursts and optional USER sidebands; complete external AXI4 endpoint compliance is not qualified. See the standard bus contracts for supported profiles and evidence boundaries. Bus-specific CDC bridges remain deferred.

8 Named module interfaces

Named module interfaces give a public name to an exact, behavior-free module signature. They let a library state the ABI that several concrete modules must implement without selecting, replacing, or instantiating any implementation.

```
interface FirIfc {
    clock clk
    reset rst
    in  x : fixed<18,16>
    out y : fixed<18,16>

    timing {
        latency 4
        ii 1
    }
}

module DirectFir : FirIfc {
    // The complete public surface is inherited from FirIfc.
    y = pipeline(4) { x }
}
```

The declaration after `interface` contains only public signature items:

- type and value parameters;
- `clock` and `reset` declarations;
- scalar, wire, ready/valid, credit, packet, and VC-credit `in`/`out` ports;
- source-authored aggregate protocol endpoints and their roles;
- at most one exact `timing { latency N ii 1 }` contract.

It cannot contain assignments, registers, rules, storage, child instances, or other implementation behavior.

8.1 Exact conformance

`module M : Ifc` means that `M` promises to have exactly the applied signature of `Ifc`. Conformance is checked during typed semantic analysis. It is not structural duck typing and does not introduce an implicit conversion.

The following all belong to the contract:

- parameter names, kinds, declaration order, and defaults;
- applied type and value parameter values after specialization;
- port names, declaration order, directions, canonical types, and protocol capacities;
- clock/reset pairs and every port or endpoint domain;
- aggregate protocol specialization, role, member ownership, payload types, and member domains;
- presence and exact value of the module timing contract.

If a conforming module declares no public surface, the complete applied interface surface is inherited: parameters, ports, clock/reset domains, aggregate endpoints, and timing. This is exact signature inheritance, not structural inference. The legacy fully redeclared form remains accepted. A partial redeclaration is never merged with the interface and fails exact conformance.

Missing or additional members in a fully redeclared surface are errors. So are renamed ports, changed directions, width changes, role changes, a different domain, and a timing contract present on only one side.

Named parameters and concrete type parameters are supported:

```
interface TransformIfc<type T, N=4> {
    in  x : vec<N,T>
    out y : vec<N,T>
}

module Transform<type T, N=4> : TransformIfc<T=T, N=N> {
    y = x
}
```

The module parameter declaration itself must match the interface parameter declaration exactly. A concrete specialization records canonical applied values in `ModuleSignature`; concise source spelling does not change that identity.

8.2 Aggregate protocol signatures

An existing source-authored aggregate protocol can be part of a named module interface. Its role and clock domain are explicit:

```
protocol TinyBus<AW=8> {
    role initiator
    role target
    channel command : rv<bits<AW>> initiator -> target
    member alarm : bit target -> initiator
}

interface TinyTarget<AW=8> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
}

module Target<AW=8> : TinyTarget<AW=AW> {
    clock clk
    reset rst
    interface bus : TinyBus<AW=AW>.target @clk
    // Concrete behavior remains in the module.
}
```

Conformance uses the already typed aggregate schema and ownership model. It does not guess compatibility from endpoint or generated RTL names.

8.3 Selection and backend identity

A named interface never chooses a concrete implementation. Source still instantiates a concrete module, and normal top selection still names a concrete module. There is no automatic substitution, inheritance, refinement search, or runtime dispatch.

The applied `ModuleSignature` is backend-independent semantic/build metadata. It survives canonical round trips, while the implementation body continues through the production Direct-SV path. Merely adding an equivalent interface declaration does not authorize a backend to change RTL or timing.

8.4 Current boundaries

The current bounded surface intentionally does not support:

- `request_response` members in named module interfaces, because their requester/responder role is currently inferred from module behavior rather than declared as an exact signature role;
- interface extension or refinement beyond exact complete-surface inheritance;
- multiple-interface conformance;
- automatic implementation selection or substitution;
- relaxed variance, implicit resizing, protocol adaptation, or CDC insertion.

Use the existing concrete module and protocol forms when one of these boundaries applies. Unsupported or non-conforming signatures fail with a structured interface diagnostic; they never silently weaken the ABI.

9 Tagged unions

ZLang tagged unions are a bounded way to carry one of several named scalar payload shapes. They are values, not procedural control flow and not protocol envelopes.

```
union Message {
    Idle
    Data { value : u8 }
    Error { code : bits<4> }
}

message : Message = Message.Data { value = input }

result : u8 = match message {
    Message.Idle => 0
    Message.Data { value } => value
    Message.Error { code } => extend<8>(code)
}
```

9.1 Type and layout

The type is nominal: two equal-looking declarations are not interchangeable. Variant order defines ordinal tag codes. The tag width is `max(1, ceil_log2(variant_count))` and occupies the most-significant bits. The payload is as wide as the largest variant. Fields occupy it from most to least significant in declaration order; a shorter payload has zero low-order padding.

The supported field types are flat `bit`, `bits<N>`, `uN`/`uint<N>`, `sN`/`sint<N>`, `fixed`, and `ufixed` fields. A fieldless value uses the concise constructor `Message.Idle`; `Message.Idle {}` remains an equivalent explicit spelling.

9.2 Construction and matching

Constructor fields are named, exact, and complete. Missing, extra, repeated, or wrongly typed fields are compile-time errors. A `match` must contain every variant exactly once. A payload arm binds every field in declaration order; binders cannot rename, omit, repeat, or shadow an existing value.

Matching is pure and zero-latency. Semantic lowering retains a typed `UnionConstruct`, `UnionTag`, and `UnionField`, then expresses selection with the ordinary typed `Switch`. The simulator carries an immutable nominal runtime value. Direct-SV carries the exact frozen packed bits. Registers and internal scalar child ports may use union values.

9.3 Deliberate boundaries

An external top-level union input is rejected because arbitrary raw bits could encode an unused tag. There is no raw decode, `bitcast<Union>`, `unpack<Union>`, generic/recursive union, nested

aggregate payload, union operator overload, wildcard/nested pattern, guard, partial match, or protocol inference in this slice. This does not change the value's arithmetic meaning.

The runnable source is `examples/tagged_union.zhl`. The exact representation and current exclusions are documented above and in Known limitations.

10 Parameterized aggregate protocols

The generic composition slice expands a parameterized protocol endpoint during semantic elaboration. For example:

```
protocol TinyBus<AW=8> {
    role initiator
    role target
    channel req : rv<uint<AW>> initiator -> target
    channel rsp : rv<uint<AW>> target -> initiator
    member irq : bit target -> initiator
}
```

A module uses this declaration with `interface bus : TinyBus<AW=8>.initiator`. The compiler retains the aggregate identity and specialization, then expands each member into the existing typed leaf endpoint IR. Ready/valid members preserve physical backward-ready propagation; plain members are ordinary scalar wires.

Top-level aggregate endpoints additionally have a stable `TopAggregateABI` projection. It recursively flattens ready/valid payload structs, derives physical direction from source/sink ownership, and keeps aggregate/member identities separate from generated names. Direct-SV consumes that projection without protocol-specific source-name guessing.

`connect left.bus -> right.bus` checks protocol identity, specialization, member names, payload types, roles, and clock domains before producing leaf hierarchical connections. Aggregate buffering, adapters, and CDC are deliberately rejected; they must be expressed on a leaf connection in a later library design.

The Direct-SV backend consumes the same closed child component ABI as other hierarchical protocol children. A child receives every scalar dependency and protocol backward signal explicitly and returns forward values plus scalar outputs. No aggregate or RTL name is reconstructed by textual substitution. Generated interface metadata records both aggregate identities and their leaf signal bindings, so formal/source attribution remains stable.

Generic value parameters support exact positive width arithmetic (`+`, `-`, `*`, and exact `/`). Type parameters are bound at an aggregate use site. The aggregate connection is structural: it does not implement AXI/APB transaction state machines, adapters, packages, CDC, or automatic protocol conversion.

The TinyBus producer/consumer example (two ready/valid channels plus a reverse scalar member) elaborates into three typed leaf connections and passes the release-tested Direct-SV lint gate.

Stateful children with multiple independent ready/valid channels are supported in Direct-SV. An unconnected aggregate bus on the selected top is projected through the shared `TopPhysical ABI` into typed public leaves. This is generic schema/ownership lowering, not an AXI semantic exception. Arrays, partial aggregate exposure, and unsupported protocol kinds are rejected.

11 Implementation intent, frequency, and II

Use `implement` when an expression defines one exact hardware value but more than one legal implementation may exist. The expression remains the required result. The nested `intent` block filters legal choices and optionally states which cost to prefer:

```
module IntentDotProduct {
    clock clk reset rst
    in a : vec<4,u3>
    in b : vec<4,u3>
    out y : u8

    y = implement {
        dot(a, b)
        intent {
            latency <= 3
            ii == 1
            dsp <= 4
            fmax >= 100
            minimize latency
        }
    }
}
```

The compiler keeps the original exact expression as a legal choice and may also consider a bounded set of equivalent structures and physical implementations. It never relaxes a hard constraint. If no candidate satisfies every constraint, compilation fails and reports the rejected choices.

11.1 `intent` clauses

An `intent` block is non-empty. Clauses are separated by whitespace or newlines; commas are not used. A metric may have at most one constraint, and the block may have at most one objective.

Form	Accepted values	Meaning
`lut REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the LUT estimate.
`ff REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the flip-flop estimate.
`dsp REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the DSP-resource estimate.
`bram REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound the block-memory estimate.
`latency REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 0`	Bound accepted-input to corresponding-output latency according to the selected contract. A positive value requires a clock/reset context.
`ii REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1`	Bound initiation interval: the nominal minimum spacing between accepted transactions when downstream capacity is available.
`fmax REL N`	`REL` is `<=`, `>=`, or `==`; `N >= 1` MHz	Bound the available frequency estimate or compatible measurement. `fmax_est` is accepted as an alias.
`minimize METRIC`	`lut`, `ff`, `dsp`, `bram`, or `latency`	Rank legal choices by one cost.
`maximize fmax`	`fmax` or `fmax_est`	Prefer the highest compatible frequency evidence.

If an objective is omitted, selection defaults to `minimize lut`. Frequency may only be maximized; resources and latency may only be minimized. II has no objective form. Constraints filter the available architectures rather than creating new ones: for example, `ii <= 4` may select an existing II=1 choice.

`pipeline(N) { expression }` is different from `implement`. It always specifies exactly `N` cycles of visible latency. An `implement` block instead allows the compiler to select among the existing

legal choices admitted by its constraints. Use `choice(...)` when the source explicitly supplies alternative expressions.

11.2 Frequency control and synthesis trials

Frequency selection and the physical synthesis clock are separate controls:

Control	Purpose
`fmax >= 100` inside `intent`	Reject an implementation choice whose admitted frequency value is below 100 MHz.
`maximize fmax`	Rank the remaining legal choices by their admitted frequency values.
`fmax = { minimum = 100 }` in a selected project profile	Apply the same minimum to profile-selected regions.
`period-ns = 10.0` under the profile clock table	Emit a real 10 ns XDC/SDC clock constraint for downstream synthesis and place-and-route.
`evidence-policy`	Decide whether estimates are accepted, measurements are preferred, or measurements are required.

For a target of `F` MHz, the corresponding physical period is `1000 / F` ns. Setting only `fmax >= F` does not create a clock constraint and does not prove timing closure. Conversely, a clock period constrains the downstream tool but does not guarantee that the design meets it.

A project profile can keep these settings together:

```
[profiles.synth_trial]
backend = "systemverilog"
backend-mode = "required"
target = "xc7z030ffg676-1"
objective = "maximize fmax"
evidence-policy = "measured_preferred"
formal-policy = "available"

[profiles.synth_trial.constraints]
latency = { maximum = 4 }
ii = { exact = 1 }
fmax = { minimum = 100 }
dsp = { maximum = 8 }

[profiles.synth_trial.platform.clocks.clk]
period-ns = 10.0
```

Emit SystemVerilog and matching constraints from that profile:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --constraints-xdc build/Top.xdc \
  --constraints-sdc build/Top.sdc \
  --exploration-report build/implementation.txt
```

Supply `Top.sv` and the appropriate constraint file to the FPGA/ASIC synthesis flow. ZLang does not silently launch a vendor tool. For a bounded generic Yosys characterization trial, request both a report and cache:

```
zlang src/top.zhl --project zlang.toml --profile synth_trial \
  --systemverilog build/Top.sv \
  --synthesis-report build/yosys-synthesis.txt \
  --synthesis-cache build/yosys-cache \
  --synthesis-target generic-lut6
```

`estimate_only` uses estimates. `measured_preferred` ranks compatible routed measurements ahead of synthesis measurements and estimates, but still permits an estimate when no compatible measurement exists. `measured_required` rejects a choice when the required compatible measurement is absent. Measurements are specific to their target, part, tool, recipe, constraints, and design; a result from a different trial is not silently reused.

11.3 Ready/valid contracts and II

For a ready/valid boundary, transactions occur only on transfers:

```
input transfer  = input.valid  && input.ready
output transfer = output.valid && output.ready
```

Latency is the unstalled distance from an accepted input to its corresponding valid output. II is the nominal minimum spacing between accepted inputs when the implementation and downstream path have capacity. Capacity is the number of accepted but not yet retired transactions the block can hold. Downstream backpressure may increase the observed wall-clock interval; it does not change the declared II.

Current support has three distinct cases:

Form	Current behavior
Ordinary scalar `implement`	Fixed-rate only. Available candidates have II=1; an II>1 constraint does not create resource sharing.
`transform pipeline(auto, ...)` over a supported ready/valid kernel	Globally stalled pipeline with unstalled II=1. Output data and `valid` hold while downstream `ready` is low.
`transform pipeline(auto) { implement { a*b + c*d ... } }`	Narrow non-interleaved shared arithmetic: one multiplier, latency 4, II 4, capacity 1.

The supported II=4 form requires one ready/valid input and output, integer operands, the exact `a*b + c*d` shape, one clock/reset domain, `dsp <= 1`, and an intent that admits II 4 and latency 4:

```
struct SharedOperands {
    a : u16
    b : u16
    c : u16
    d : u16
}

module SharedMultiply {
    clock clk reset rst
    in input : rv<SharedOperands>
    out output : rv<u33>

    input -> output {
        transform pipeline(auto) {
            implement {
                input.payload.a * input.payload.b
                    + input.payload.c * input.payload.d
                intent {
                    latency <= 4
                    ii == 4
                    dsp <= 1
                    minimize dsp
                }
            }
        }
    }
}
```

Only one transaction is active. While computation or an unconsumed result is present, new input is blocked. When `output.valid && output.ready` retires a result, the block may accept the next input on the same edge; this is why the `RETIRE_AND_RELOAD` schedule has II 4 rather than II 5. A stalled output remains valid with stable payload, and input readiness remains low until retirement is possible.

This is not general HLS. It does not support overlapping transactions, modulo scheduling, arbitrary arithmetic graphs, more products, fixed-point temporal sharing, memories, CDC, or multiple active transactions. The transaction-stream formal route supplies bounded model-checking evidence for this exact case. `required_bmc` may accept it only after `bounded_pass`; `required_proven` rejects it because no unbounded proof is claimed.

The runnable example is `examples/temporal_shared_multiply.zhl`.

12 Optimization and formal verification

12.1 Canonical and selected IR

Compilation retains a high-level typed representation and a selected architecture representation. Metadata includes canonical type, width, signedness, latency, initiation interval, domain, purity/effects, source origin, and separate estimated/measured cost evidence.

The pure e-graph layer uses the dependency-pinned egglog engine for a bounded, type-safe scalar rewrite subset. It covers exact integer/fixed arithmetic, bitwise/shift, compare/mux, resize and wiring nodes. Fixed conversion is an opaque quantization boundary. Reassociation of ordinary carry-growing adds, movement across rounding/rescale, state, timing, protocols, storage, rules, and CDC are excluded. Source `equiv` declarations register only accepted exact same-cycle value rules; they are not assertions or temporal equivalence. See Optimization and evidence boundaries for the distinction between exact value alternatives, legal implementations and executed evidence.

12.2 One implementation-policy path

The staged flow is explicit; egglog is a typed value-alternative producer, not the pipeline or architecture search engine:

```
typed value IR
    -> optional egglog pure-value alternatives
    -> typed computation DAG
    -> bounded generic/resource covering
    -> target-aware exact-N fixed-latency scheduling
    -> deterministic cost selection
    -> optional formal-aware semantic-reference proof gate
```

Timing alignment validates latency/II for eligible candidates. That validation is not, by itself, a formal proof.

The arithmetic exploration tutorial demonstrates an exact eight-product expression, topology-only selection versus an internally registered pipeline, real latency-aware equivalence/mutations, and independent FPGA timing measurement. Solver success and estimated frequency must not be reported as routed 100 MHz timing closure.

The canonical source form is `implement`; all compiler-selected forms normalize through the same typed implementation-policy/extraction infrastructure:

```
y = implement {
    dot(a, b)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        minimize lut
    }
}
```

`implement` first performs bounded exact-equivalence exploration for its one pure eligible value root. The source form is always retained. Checked egglog alternatives are deduplicated by a target-neutral structural signature (typed operation DAG, operation counts, depth and fanout), then the limited set of structurally distinct values is presented to the ordinary implementation providers. Constraints still filter complete implementation candidates and the objective still ranks only legal survivors; egglog does not see DSP, FF, latency, II, target or protocol costs.

The default compiler policy permits at most six saturation iterations, 4096 typed e-classes, 16384 e-nodes, sixteen raw extractions, eight retained value structures, 32 provider candidates per retained structure and 64 candidates in the complete local catalog. These are deterministic compiler work bounds, not language syntax or an optimality claim. Crossing an e-graph size bound keeps the independently checked source structure and records the limitation in the exploration report; it never admits an unchecked value or relaxes intent. Projects may lower these bounds, or raise them within compiler hard ceilings, through the selected profile's `intent-exploration` table. No wall-clock timeout participates in normal selection, so the retained subset stays deterministic. Identical exact roots reuse a bounded session/workspace structural recipe, but current origins are rebound and exact certificates are replay-checked on every use. Intent filtering, target selection, formal evidence, and the final chosen implementation remain fresh computations.

`implement` then selects applicable bounded II=1 structural CSA multiplier candidates for exact uniform integer products up to 16 bits per operand, reduction/DSP, and (when a legal clock/reset context exists) fixed-latency pipeline candidates. It does not enable unsafe reassociation, CDC, protocol adaptation, general variable-II sharing, or general retiming implicitly. Those remain compatibility/profile-controlled features with their existing legality checks.

The remaining source forms are:

- `choice(...)` for explicit equivalent arms and bounded cost selection;
- protocol `transform pipeline(auto, ...)` for the existing globally-stalled ready/valid transform.

The scalar spellings `architecture(auto)`, `pipeline(auto)`, and `explore` are retired and fail with migration diagnostics.

`intent` metric	Accepted condition	Meaning / limit
`lut`, `ff`, `dsp`, `bram`	`<=`, `>=`, `==` nonnegative integer	Hard estimated resource bound; not proof of physical usage.
`latency`	`<=`, `>=`, `==` nonnegative integer	Observable sample latency; positive bounds can enable legal clocked pipeline candidates.
`ii`	`<=`, `>=`, `==` positive integer	Initiation interval of an existing candidate. `throughput` is a compatibility alias in implementation profiles.
`fmax` / `fmax_est`	`<=`, `>=`, `==` positive MHz value	Estimated frequency bound; target evidence rules still apply.
`minimize`	`lut`, `ff`, `dsp`, `bram`, `latency`	One objective; omitted objective defaults to `minimize lut`.
`maximize`	`fmax_est` (or `fmax`)	One frequency objective; `maximize` cannot target resource/latency metrics.

An `intent` needs at least one clause; there can be at most one constraint per metric and one objective. Constraints filter candidates, then the objective ranks survivors. Unsupported metrics, contradictory contributions and an empty candidate set are errors, not relaxed requests. `ii == 4` does **not** cause four-cycle sharing of one DSP: current scalar source/pipeline candidates have II=1. `ii <= 4` may therefore select an II=1 candidate without reducing DSP count. To request a smaller DSP estimate, use a `dsp` bound/objective and inspect the selection report. The sole exception is the explicitly flow-controlled foundation described below; ordinary scalar `implement` never gains transaction state or II>1 sharing.

The tested II boundaries are easier to read as three separate contracts:

- **Scalar `implement`.** Structural multiplier, reduction/DSP, and legal fixed pipeline alternatives currently have `ii=1`; selection tests check the reported interval and constraints. This does not imply automatic multi-cycle DSP reuse or a routed Fmax guarantee.
- **Ready/valid `transform pipeline(auto, ...)`.** The supported kernel accepts one beat per unstalled edge after fill, while a stall holds the entire pipeline. Simulation and Verilator cover bubbles and backpressure. This does not promise one beat per edge while `ready=0` or independent per-stage elasticity.
- **Bounded shared arithmetic.** For `transform pipeline(auto) { implement { a*b + c*d intent { ... } } }`, the compiler may choose one non-interleaved multiply resource with `ii=4`, latency 4, and capacity 1; output retirement may capture the next input on the same edge. This does not add modulo scheduling, transaction interleaving, arbitrary-DAG scheduling, memory sharing, or general HLS.

An II metric is a candidate contract, not a request to synthesize a new time-multiplexed architecture. For actual resource use and timing, inspect the emitted implementation and downstream synthesis/routed reports.

Hard constraints are never silently relaxed. Estimated cost and measured synthesis evidence stay distinct. Automatic protocol adaptation, CDC insertion, general retiming, variable latency, and II-changing sharing are not inferred.

12.2.1 Explicit choices

```
y = choice(auto, minimize=lut, dsp<=1, latency<=1, ii<=1) {
    mul_add => pipeline(1) { a * b + c }
    dsp_mac => pipeline(1) { a * b + c }
}
```

`dsp_mac` is mapping intent, not evidence that a physical DSP was used.

12.2.2 Exact and selected timing

```
y = implement {
    a * b + c * d + e * f + g * h
    intent { latency <= 3 ii == 1 dsp <= 4 fmax >= 100 }
}
```

`implement` considers pipeline candidates only in a legal clock/reset region when the intent explicitly permits positive latency. `pipeline(3) { expr }` remains exact three-cycle hardware semantics and is never lowered to an implementation preference.

For supported pure scalar DAGs the physical record is one `ScheduledValueGraph`: operations, exact stage assignment, generic or resource bindings, resource-local/fabric cuts, balancing delays, latency, II and cost provenance. Egglog never places these cuts. On Xilinx 7-Series the target planner can cover standalone multiply, MAC/add-sub, preadd-multiply and ordered signed-product cascades with DSP48E1; uncovered operations remain fabric. Unknown target timing cannot satisfy an Fmax constraint, and structural cost is never reported as a measurement.

Only frozen expression shapes are accepted. Candidate timing is checked and the result is a concrete fixed-latency expression. Target-aware fixed-FIR planning is similarly bounded and keeps final fixed-point quantization outside partial products/reduction nodes.

The first protocol-aware form is an explicit ready/valid transform:

```
input -> output {
    transform pipeline(auto, latency<=3, ii==1, fmax>=100) {
        input.payload.a * input.payload.b
          + input.payload.c * input.payload.d
          + input.payload.e * input.payload.f
          + input.payload.g * input.payload.h
    }
}
```

This form **works** for the tested `ElasticPipelineAuto` example: semantic simulation checks first beat, bubbles, reset and global stall, and generated SystemVerilog is exercised with Verilator. It accepts one ready/valid input and output in one synchronous domain and a pure supported product-reduction kernel. All data registers and the valid chain share one global clock enable: they advance only when the output is empty or ready.

Property	Supported contract
Unstalled latency	Selected fixed value `L`.
Unstalled initiation interval	`1` (`ii_no_stall=1`).
Capacity	`L` beats.
Backpressure	Wall-clock latency may grow; output data/valid hold under stall.
Not supported	Independently elastic stages, user registers/rules/storage, CDC, adapters or protocol-control captures inside the kernel.

This is a bounded protocol transformation, **not** a general automatic pipeline/retiming feature. Physical DSP binding additionally needs a target resource compatible with the transform's clock-enable/stall behavior; `preferred` may use generic RTL and `required` rejects an unsupported binding.

12.2.3 Capacity-one temporal sharing foundation

The first II>1 candidate is intentionally available only *inside* a typed ready/valid transform, never for an ordinary wire/scalar `implement`. Its current accepted shape is exactly two uniform integer products followed by one integer addition:

```
input -> output {
    transform pipeline(auto) {
        implement {
            input.payload.a * input.payload.b + input.payload.c * input.payload.d
            intent { latency <= 4 ii <= 4 dsp <= 1 minimize dsp }
        }
    }
}
```

`II` is an accepted-transaction capability, not a decorative `ready` waveform. After `input.valid && input.ready`, this capacity-one implementation blocks admission while its three scheduled operations run and while its result is pending. On a prior `output.valid && output.ready` retirement it may capture the next input on that same edge; under output backpressure, `valid` and payload hold stable and `input.ready` remains low. Latency is the unstalled accepted-input-to-output-valid distance; capacity is separately recorded as one.

The semantic e-graph owns **what** exact value may be computed; the temporal scheduler owns **when**, resource binding owns **where**, and transaction equivalence is separate. A capacity-one transaction miter checks accepted inputs against retired outputs, including same-edge retirement and reload, in bounded formal runs. The route is bound through the selected `Backend Artifact`; `available` records advisory evidence and `required_bmc` requires `bounded_pass`. It is not an unbounded proof: `required_proven` rejects explicitly. Fixed-latency scalar equivalence is never reused for this temporal candidate.

This public-alpha foundation has a deliberately hard boundary.

Supported:

- one ready/valid input and one ready/valid output in the same clock/reset domain;
- the exact integer kernel `a*b + c*d`;
- one shared multiplier, non-interleaved execution, capacity one, latency four, and II four;
- same-edge output retirement and next-input capture under normal ready/valid transfer rules;
- transaction-stream BMC as bounded candidate-selection evidence.

Unsupported:

- scalar/fixed-rate II>1 `implement` regions;
- overlapping transactions, modulo scheduling, arbitrary operation DAGs, or more products;
- memory or CDC sharing, fixed-point temporal sharing, and nested protocols;
- `required_proven` or any claim of unbounded temporal proof.

The checked public example is `examples/temporal_shared_multiply.zhl`.

Target and resource descriptions under `std.target.*` and `std.arch.*` are compiler-shipped source. Functional modules do not name vendor registers or primitives. Manual `required`, `preferred`, and `generic` architecture modes determine failure/fallback behavior; selecting a target alone does not start automatic search.

12.3 First-class verification goals and contracts

Verification declarations are a verification-only overlay over the typed module. Standalone safety and bounded-reachability goals are named:

```
assert count_within @ clk {
    count <= DEPTH
}

cover reaches_done @ clk {
    state == State.Done
}
```

Related requirements and goals can share a contract scope:

```
contract fifo_behavior @ clk {
    require legal_input {
        input_length <= 1500
    }

    assert count_within {
        count <= DEPTH
    }

    ensure output_shape {
        !output.valid | output.payload.ok
    }

    cover full {
        queue.full
    }
}
```

`assert` and `ensure` are same-cycle safety obligations. `cover` is bounded reachability and never constrains the design. `require` is an environment-owned precondition local to its contract; all requirements in one scope are conjoined and gate that scope's goals. An `ensure` must observe at least one implementation-owned public output and cannot depend on hidden state. Goal/clause

names are mandatory and unique. `@ clk` may be omitted only when exactly one clock/reset domain exists; reset is inferred from that domain and suppresses sampling.

The existing forms remain source-compatible:

```
assume bounded @clk disable iff rst {
    (a < 8) & (b < 8)
}

guarantee sum_matches @clk disable iff rst {
    y == a + b
}
```

Legacy `assume` normalizes into a module-global requirement and `guarantee` into a module-global assertion. Every body has ordinary ZLang type `bit`. Cross-domain references and unsupported temporal forms are rejected. This slice does not add arbitrary SVA/SMT, liveness, eventual delivery, fairness, or temporal implication syntax.

An `assume` may reference environment-owned leaves only. In particular, a ready/valid or request/response `.transfer` combines an environment input with an implementation output and is therefore not a legal assumption predicate. `guarantee` remains free to observe both sides. Executable declarations lower to typed structured predicates; report strings and RTL names are never parsed to recover their meaning.

The simulation `VerificationMonitor` samples after combinational settle and before edge commit. Failed active `assert`/`ensure` goals are source-attributed DUT failures. Violated `require`/`assume` clauses are recorded as environment violations and gate their dependent goals. Cover records only its first witness cycle; a missing simulation witness is not a failure.

`--contracts-sva` emits supported bindable safety goals through the same structured predicate meaning as safety verification. Missing observations remain explicitly non-executable rather than falling back to a second expression walker.

Each non-empty assumption set has a feasibility cover. A compile-time false requirement is rejected; a dynamically unwitnessed requirement makes dependent safety success `unknown` with a vacuity diagnostic rather than a pass.

Executable formal routes consume the exact typed physical-domain contract. With `power_up unspecified`, safety verification/source safety and cover, existing bindable recursive safety verification observations, fixed-latency II=1 semantic-reference equivalence, and the corresponding formal-aware selection policies support rising/falling edges, synchronous or raw asynchronous assertion, both polarities, and the existing two-active-edge synchronized release. Asynchronous formal execution remains single-domain; multiple synchronous domains may still produce independent goal-local jobs. Same-cycle pure candidate equivalence remains reset-independent.

The checker normalizes external polarity to one active-high effective reset. For synchronized release, checker history, property guards, feasibility covers, fill masks, and comparison windows stay reset-masked for both release edges after the raw pin deasserts. Formal traces publish both `physical_reset` (the raw external pin) and `trace:reset` (the effective reset); result `reset_state` is the effective view. Unsupported contracts are never coerced to the legacy reset model.

12.3.1 Execution and immutable bundles

Start with the runnable verification tutorial for a proved state invariant, an intentionally rare overflow counterexample, scoped assumptions and RV stall checking. It includes the important case where shallow BMC passes a broken design and deeper replay finds the defect.

```
zlang design.zhl --verify
zlang design.zhl --verification-bundle build/verify
zlang verify build/verify
```

`--verification-report PATH` plus `--verification-format text|json` publishes a structured result. `--verify-require checked|proven` selects the requested safety level. `--formal-jobs N`

controls independent safety/cover bundle jobs and independent selected-candidate sites. Dependencies inside one candidate site remain ordered and report ordering remains deterministic. `--verification-work-dir DIR` and replay's `--work-dir DIR` retain generated SBY inputs, logs, and VCDs outside the immutable bundle. A bundle contains a hash-validated manifest, structured verification IR, implementation/formal artifact, source map, separate safety/cover harnesses, and exact ROM companions when needed. Engine, solver, depth, timeout, tool versions, logs, and results belong to replay execution rather than the immutable source bundle identity; the replay tool regenerates execution-specific SBY configuration.

The immutable bundle and separate run report retain deterministic identity, route provenance, staged BMC/prove evidence and per-job work/tool attribution. Joint compiler execution also links candidate semantic-reference equivalence evidence. Retained VCD frames are mapped through the bundle bindings to semantic signal IDs for source-facing witness and counterexample values. Work paths and raw logs remain reproducibility data, not semantic or run identity.

A `proven` request is never a direct shortcut to prove mode. BMC executes first at the requested depth; every safety job must be `bounded_pass` before those safety jobs enter prove mode. Covers execute once and are not rerun. An unrelated cover miss/skip does not block proof, while a feasibility-cover miss first makes its dependent safety job `unknown` and therefore blocks proof.

Safety statuses remain `failed`, `bounded_pass depth=N`, `proven`, `unknown`, and `skipped`. `bounded_pass` is never proof. Cover has a distinct vocabulary: `witnessed cycle=N`, `bounded_unreached depth=N`, `unknown`, and `skipped`. `bounded_unreached` is neither `proven` nor `unreachable`, and an ordinary cover miss does not make the command fail.

Exit status `0` means the requested safety level was satisfied. A safety or source counterexample, or an executed semantic-reference equivalence counterexample, returns `1`. Unknown, skipped, or vacuous verification; tool/configuration failure; or a requested proof with only bounded evidence returns `2`. A cover miss alone is not a failure, and unavailable advisory candidate evidence does not alter formal-aware selection eligibility.

12.4 Formal layers

Variable-latency elastic ready/valid transforms are deliberately outside the semantic-reference equivalence fixed-latency relation. They never enter that route by treating their minimum unstalled latency as wall-clock latency.

The one capacity-one `a*b+c*d` temporal candidate instead has a separate transaction-indexed BMC route. Its proof recipe binds the relation identity, implementation identity, exact direct-SV artifact and bindings, harness, depth, tools, and reset contract. This route may satisfy `required_bmc` only with `bounded_pass`; it cannot satisfy `required_proven`.

Decisive formal-aware selection results are route-bound data, not trusted callback booleans. The property, harness, assumptions, backend route, semantic-reference artifact, implementation artifact, engine, mode, and depth must all match the cache identity before a candidate can become eligible. Exact reuse additionally matches stage policy, timeout, tool snapshot, dependencies, and compiler schema. Failed results require typed counterexample metadata; non-failed results reject it. Decisive semantic-reference equivalence evidence requires a positive depth. Missing `yosys-smtbmc` is reported like any other genuinely unavailable formal tool and never becomes false success. An `unknown` result or timeout in a required mode terminates the selection: the compiler never lets solver runtime become an implicit architecture objective.

`bounded_pass` means no counterexample was found through the stated BMC depth; it is not an unbounded proof. Only `proven` satisfies `required_proven`. `failed`, `unknown`, and `skipped` do not satisfy required policies.

Semantic analysis creates candidate spaces but runs no backend or solver. A compiler-owned candidate-site ledger carries stable site identity and exact rank into the selection phase, where `--formal-policy` executes the formal-aware selection route. Bundle-only publication records one strict compiler-owned linking plan over the exact verification goal plan, candidate-site ledger, and retained formal-aware selection records. Joint `--verify` with a non-off policy enriches that immutable plan with the selected-candidate semantic-reference equivalence plan before execution; candidate results are emitted afterward in the combined report/evidence. Bundle publication may additionally freeze the exact selected-candidate semantic-reference equivalence inputs as hash-validated, path-free companions; `zlang verify` then executes those

inputs without source compilation or formal-aware selection reselection. Base bundles contain no candidate replay inputs. The common evidence report validates those links without merging result types. For an executed BMC route it publishes the exact status, route, depth, property or relation identity, solver, known tool versions, and `unbounded: false`. Reset contracts and assumptions are serialized only when the compiler-owned route already represents them; the report does not infer missing context. Capacity-one temporal evidence also records the `capacity_one_transaction_stream` relation, latency 4, II 4, capacity 1, same-edge retire/reload, and that `required_proven` is unsupported. A solver matrix preserves one shared immutable problem identity and independent per-solver run identities/results, with disagreements represented explicitly; agreement never upgrades the underlying result status. Repeated candidate implementations are associated by semantic site and rank, not by candidate identity alone. With policy `off`, the ledger remains visible but there are no formal-aware selection attempt/evidence records. Candidate execution uses deterministic recipe-addressed work roots below the external verification work directory. An exact in-session formal-aware selection reuse retains its recorded work root; persistent formal-aware selection cache data deliberately excludes physical paths, so a cache hit never claims that an old workspace still exists. These paths and the discovered candidate tool snapshot are operational report data, not proof, cache, or compiler-verification-report identity. Parallel sites with an identical proof recipe share the same provider-owned workspace and retain the same complete attribution. `--formal-harness` and `--formal-sby` remain separate safety verification artifact-generation options; writing either file alone is not proof execution. An incomplete or mixed-route compatibility view is rejected with guidance to use a verification bundle rather than silently choosing one backend.

The current bounded applicability also transports observations already required by existing properties: parent request/response outstanding and directional-buffer counts, receiver-credit adapter occupancy, and direct same-domain internal ready/valid guarantees. These are formal-only typed ABI ports or root-monolithic assertions, never inferred RTL names or substituted environment assumptions. Automatic root assumptions receive a separate feasibility cover, so dependent safety evidence is vacuity-checked.

Range-proven runtime vector selection is available to the existing same-cycle predicate IR. One separate compiler-owned helper can derive whole-root same-cycle semantic-reference equivalence evidence for exactly one pure scalar child by following typed hierarchy bindings and producing formal-only Yosys namespaces. It does not flatten production RTL and does not authorize state, protocols, storage, arrays, nesting, or aggregate boundaries. Protocol-valued and aggregate-protocol top boundaries fail closed in the scalar semantic-reference equivalence entry points.

See the language support matrix and Known limitations for the accepted matrix and remaining semantic boundaries.

12.5 Reports

Useful current outputs include high-level and selected optimization IR, saturation, implementation, cost, pipeline, architecture, exploration, synthesis, artifact manifests, and structured formal results. See Check and run ZLang for commands and feature-specific design records for the exact schema.

13 E-graph and optimization responsibilities

An exact value alternative is not a complete hardware implementation or a formal proof. Bounded value exploration preserves typed binary behavior; implementation selection must additionally satisfy the requested timing, resource and evidence policy. It does not imply exhaustive optimization or general HLS.

Widths, signedness, truncation, fixed-point scaling, rounding and saturation remain explicit numerical boundaries. State, reset, clocks, protocols and CDC are not pure-value rewrite domains. A transformation crossing a numerical boundary needs a specifically supported exact rule; an unsupported case is rejected rather than approximated.

Target mapping supports only the documented resource capabilities, including the admitted Xilinx 7-Series DSP48E1 subset. A multiply-add spelling alone does not guarantee a DSP binding.

Structural estimates are not synthesized results or routed timing; missing timing evidence never means zero delay.

A checked value-equivalence certificate establishes only its typed value relation. Actual-RTL evidence must cover the selected implementation and its recorded contract separately. `bounded_pass`, `unknown` and `proven` remain distinct; unavailable or incomplete evidence cannot satisfy `required_proven`.

14 Storage-owning instance arrays

This bounded composition slice extends one-dimensional compile-time instance arrays to same-domain scalar-wire children that own one storage resource. It does not add source syntax: the existing `inst lane[N]`, indexed bindings, and indexed child-output references elaborate into the existing typed hierarchy.

14.1 Supported boundary

A child specialization may contain ordinary registers/rules without storage, or own at most one of:

- a FIFO;
- a synchronous memory; or
- an initialized ROM.

The legacy globally controlled storage profile remains storage-only: it does not combine that resource with user registers, next-state assignments, or rules. The separately validated scheduled-FIFO profile does permit ordinary register/rule state. FIFO actions and register writes are selected atomically by the same existing `ResolvedTransition`; the array backend does not create a second scheduler. Unsupported ownership combinations fail during semantic analysis rather than reaching a backend that could emit only part of the child.

The child must retain the existing storage-array contract: scalar wire ports, one inherited clock/reset domain, and no nested instances. Multiple scalar outputs are allowed. Each array element has a distinct physical instance identity and storage state; all elements share the one deterministic specialization identity. The Direct-SV backend consumes `ElaboratedInstance` bindings directly and emits one reusable component plus one application/instance per physical element.

CSR, other protocol/storage mixtures, multiple storage resources, arbitrary nested storage hierarchy, multiple clock domains, CDC, runtime-selected inputs or protocol endpoints, and formal observation extensions remain rejected. Runtime selection of a bit-packable scalar-wire output is supported as a read-only projection: `lane[select].value` lowers to a typed runtime index over all statically elaborated `lane[i].value` references. Every child continues to execute and retain independent state; the selector is only an output mux and never gates a child or denotes a dynamic physical instance.

The one bounded protocol/storage exception is a storage-only child whose external ports are exclusively primitive ready/valid and which owns exactly one globally controlled FIFO. It uses the same typed `HierarchicalConnection`, closed component ABI, and FIFO state as a non-array child. Ready/valid arrays with synchronous memory or initialized ROM remain rejected because they do not yet have a frozen protocol/storage contract.

The two-lane synchronous-memory witness checks independent cells and addresses, write-first same-address collision behavior, registered read output, and reset clearing both cells and read state. It passes strict Direct-SV/Verilator behavior.

The `ZtpuBankedMemory` witness elaborates two four-element arrays into eight independently identified physical memory children with one shared specialization. Runtime output projection selects the addressed bank separately for each read port; decoded writes are broadcast to the matching bank in both replicas. The semantic simulator and Direct-SV/Verilator agree on masked writes, independent reads, `read_first` collisions, reset suppression/preservation, and post-reset contents. The versioned BackendArtifact records every leaf path and no fictitious dynamic instance. Hidden cells gain no new formal observation family.

15 Source and product identity

The public product name is **ZLang HDL**. Its distribution and repository are `zlang-hdl`; the compiler command is `zlang`, the source suffix is `.zhl`, the VS Code language ID is `zlang-hdl`, and the MIME type is `text/x-zlang-hdl`.

The `.zl` suffix is not a compatibility alias. Physical compiler inputs using it fail with diagnostic `ZL-SOURCE-EXTENSION`; use `.zhl`.

Logical imports do not contain a source suffix. For example, `import std.math.fixed` resolves to `stdlib/math/fixed.zhl`, while project and locked-package imports use their logical module identities.

Older dependency locks are rejected by the current versioned lock schema; regenerate them with `zlang lock update`. Source, build, and proof caches are bound to dependency and source-unit identities, and stale inputs miss safely.

The short prose name **ZLang** remains valid after the product has been introduced. The Python package `zlang`, `zlang.toml`, `zlang.lock`, the compiler-owned `.zlang/` state directory, environment variables prefixed `ZLANG_`, generated HDL identifiers, and the TextMate scope `source.zlang` are intentional product identities.

16 Language support matrix

Canonical implementation-selection markers: `implement`, `choice`.

Implementation selection uses the canonical `implement` form for compiler-discovered candidates and `choice` for user-supplied alternatives. The retired scalar `pipeline(auto)`, `architecture(auto)`, and `explore` spellings are migration diagnostics; only protocol `transform` `pipeline(auto, ...)` remains source syntax.

`zlang.public_capabilities.CAPABILITY_REGISTRY` is the machine-readable source for public capability context, semantic status, backend-independent IR, simulator/Direct-SV/formal coverage, executable witness, and current limitations. The advertised witnesses are compiled by the public capability gate. `examples/all_syntax.zhl` is a representative language tour, not a substitute for that phase-specific capability record.

The following language/backend view is generated from that registry. `bounded` means the detailed limits in this reference and the capability record still apply; “supported” is not a promise for every composition. `Simulation` is the registry's current generic simulator field, not a separate native-JIT guarantee. Native execution fails explicitly for unsupported plans; see Native simulation. Formal entries name the specific eligible relations rather than promising arbitrary proof.

16.1 Language and execution

Capability	Language	Simulation	Direct-SV
`scalar-datapath`	supported	supported	supported
`exact-literals-and-packed-constants`	supported	supported	supported
`fixed-point`	supported	supported	supported
`aggregates`	supported	supported	supported
`characters-strings-tuples`	bounded	supported	supported
`tagged-unions`	bounded	supported	supported
`functional-datapath`	supported	supported	supported
`compile-time-generation`	supported	not applicable	supported after specialization
`concise-exact-lowering`	supported	supported	supported
`typed-static-parameters`	bounded	not applicable	supported after specialization
`generic-rom-and-table-gather`	bounded	supported	supported
`sequential-state`	supported	supported	supported
`registered-outputs-and-drive`	supported	supported	supported

Capability	Language	Simulation	Direct-SV
physical-clock-reset	bounded	supported	supported
multi-clock-stateful-logic	bounded	supported	supported
encoded-enums-and-fsm	supported	supported	supported
vector-state-update	bounded	supported	supported
fifo-storage	supported	supported	supported
writable-memory	bounded	supported	supported
ready-valid	supported	supported	supported
credit	supported	supported	supported
request-response	supported	supported	supported
aggregate-protocols	bounded	supported	supported
ahb-lite-stdlib	bounded	supported	supported
cdc	bounded	supported	supported
csr	bounded	supported	supported
contracts	supported	supported	artifact generation
exploration	bounded	not applicable	selected candidates
bounded-intent-structural-exploration	bounded	not applicable	selected typed alternatives
capacity-one-temporal-sharing	bounded	supported	supported
elastic-ready-valid-pipeline	bounded	supported	supported
per-bit-native-logic-state	bounded	supported	unchanged
combinational-instance-arrays	bounded	supported	supported
runtime-instance-output-projection	bounded	supported	supported
sequential-instance-arrays	bounded	supported	supported
storage-instance-arrays	bounded	supported	supported
ready-valid-instance-arrays	bounded	supported	supported
request-response-instance-arrays	bounded	supported	supported

16.2 Formal coverage

Capability	Formal coverage
scalar-datapath	same-cycle equivalence where eligible
exact-literals-and-packed-constants	same-cycle equivalence where eligible
fixed-point	fixed-point equivalence within supported widths and forms
aggregates	value equivalence where all required fields are observable
characters-strings-tuples	scalar and packed-value equivalence where eligible
tagged-unions	no dedicated tagged-union proof support
functional-datapath	scalar and fixed-point equivalence where eligible
compile-time-generation	compile-time only; formal execution not applicable
concise-exact-lowering	same formal coverage as the equivalent explicit source form
typed-static-parameters	not a runtime formal target; specialized values participate in the elaborated design being checked
generic-rom-and-table-gather	storage safety checks only; no dedicated table-gather proof
sequential-state	register and rule safety checks where the required state and events are observable
registered-outputs-and-drive	registered outputs inherit state safety checks where observable; transient drive actions have no separate proof route
physical-clock-reset	reset-aware properties for supported edge, polarity, assertion, and release modes when power_up is unspecified
multi-clock-stateful-logic	source goals are checked in their declared domains; automatic safety checks are bounded
encoded-enums-and-fsm	no dedicated enum or FSM proof support

Capability	Formal coverage
`vector-state-update`	register safety where the selected element state is observable
`fifo-storage`	bounded FIFO safety checks
`writable-memory`	no dedicated memory-content proof support
`ready-valid`	payload-stability and transfer safety where observable
`credit`	sender and receiver credit safety when the exact counter is observable
`request-response`	outstanding-transaction and directional-buffer safety where observable
`aggregate-protocols`	no general aggregate-protocol proof support
`ahb-lite-stdlib`	register-state and ready/valid safety where signals are observable
`cdc`	no CDC correctness proof
`csr`	CSR state and access safety checks where observable
`contracts`	safety checking and bounded cover execution when supported
`exploration`	candidate equivalence can gate required selection when a supported equivalence check is available
`bounded-intent-structural-exploration`	exact-value alternatives retain equivalence provenance before implementation evidence is evaluated
`capacity-one-temporal-sharing`	capacity-one transaction-stream bounded model checking; unbounded proof is not claimed
`elastic-ready-valid-pipeline`	ready/valid stability safety only; end-to-end equivalence is not supported
`per-bit-native-logic-state`	simulation-only diagnostic semantics; not a formal proof relation
`combinational-instance-arrays`	hierarchical equivalence is not supported
`runtime-instance-output-projection`	hierarchical equivalence is not supported
`sequential-instance-arrays`	bounded safety for exposed child state and events; no general recursive hierarchy proof
`storage-instance-arrays`	no formal access to unexposed internal storage contents
`ready-valid-instance-arrays`	ready/valid safety where the required signals are observable
`request-response-instance-arrays`	request/response safety where the required signals are observable

17 Known limitations

This chapter describes the public `0.1.0a24` candidate surface.

ZLang HDL rejects unsupported combinations instead of generating partial RTL. The following restrictions are part of the current user-facing contract.

17.1 Language and RTL boundaries

- ZLang has no runtime procedural `if`, mutable local variables, implicit numeric casts, implicit fixed-point rescaling, or general HLS scheduler.
- Integer `%` is compile-time only. Runtime and fixed-point remainder are unsupported; ports, registers and ordinary function arguments are not compile-time operands.
- Runtime-selected instance inputs/protocols, general cross-module atomic scheduling, fully qualified AXI4 endpoint compliance, automatic CDC insertion, and arbitrary stateful elastic pipelines are unsupported. Bounded source-owned AXI4 profiles with IDs and bursts are available under validation; consult the standard bus contracts rather than assuming complete endpoint compliance.
- Stateful objects inside one module may belong to independent explicit clock domains. A bounded `async_mem` is the sole storage exception: its one writer and one registered reader

have distinct owners and form an explicit semantic boundary. Asynchronous 2RW memory, mixed widths, automatic banking, cross-clock atomic rules, derived/gated clocks, and target-aware scheduling across a CDC boundary are unsupported. Ordinary global memory control cannot be combined with unrelated user register/rule state in the same module; use hierarchy for that composition.
- Unsupported Direct-SV combinations produce a diagnostic rather than partial RTL.

17.2 Other bounded combinations

The former combined guide's detailed exclusions remain part of this contract:

- Packed indexing and fixed-width `value[offset +: WIDTH]` need an unsigned index with a proven range. Variable-width/reversed slices, runtime reshape, heterogeneous-vector concatenation, enum `bitcast`/`pack`/`unpack` and external top enum inputs are unsupported.
- Traits, runtime polymorphism, generic methods and recursive generic programming are unsupported. Nominal overloads retain their actual typed bodies; the compiler does not assume algebraic laws from an operator symbol.
- Dynamic vector updates admit one range-proven element of a one-dimensional `reg vec<N, T>`, not arbitrary nested writes/ranges. Read-only output projection such as `lane[select].value` is distinct from selecting an instance input.
- ROM has no reloadable/asynchronous/multiport form or automatic exploration. Writable memory admits uniform compile-time initialization, not arbitrary per-address images. Named same-domain memory ports are bounded to eight; banking and arbitration remain explicit architecture.
- There is no package registry, mutable dependency revision or unlocked external project import. A standalone source file with no project/dependency manifest is a different, supported workflow.
- Named module interfaces check exact conformance; they do not automatically substitute implementations. Arbitrary cross-module implementation selection is not implied by the existence of an interface.
- General liveness/eventuality/fairness, arbitrary solver-specific temporal source properties and general hierarchical semantic-reference equivalence remain outside the supported proof relations.

17.3 Verification boundaries

Formal support is specific to each property family and observation boundary. `bounded_pass` is not an unbounded proof, and `skipped`, `unavailable`, and `unknown` make no proof claim. Report any accepted design that emits invalid RTL as a correctness defect.

18 ZLang HDL concise source-authoring reference

This page is a concise guide to writing and running ZLang HDL. Source-language rules define the hardware; generated SystemVerilog is an output format, not a second language specification.

Full details: Language Reference, Standard Library Guide, User Guide.

For an unfamiliar construct, consult the detailed language chapter and known limitations. For exact width tables and supported forms, follow the linked topic guide.

18.1 Non-negotiable rules

- Source files use `.zhl`. The old `.zl` suffix is rejected.
- Run `zlang SOURCE --check`; highlighting is lexical and is not validation.
- Native simulation needs a matching platform wheel and fails explicitly when it is unavailable.
- Combinational connections such as `y = a + b` are continuously defined; state updates such as `count <- next_count` schedule next state at the active clock edge. The `=` token also appears in immutable bindings and initializers; it does not imply a state update.
- Assignment types are exact. There is no implicit resize, signedness change, fixed-point rescale, protocol adapter, or clock-domain crossing.

- ZLang has no `let`, `const`, `var`, `return`, semicolons, C-style `input`/`output`, or runtime procedural `if` syntax.
- `if` is compile-time elaboration only. Use `?:`, `mux`, or `switch` for pure runtime values; use `when`/`else` `when`/`else`, `priority`, or `fsm` for state actions.
- Scalar `pipeline(auto)`, `architecture(auto)`, and `explore` are retired. Compiler-selected scalar implementation uses `implement`.
- `pipeline(N) { expression }` is exact N-cycle hardware, not an optimization request. Protocol `transform` `pipeline(auto, ...)` is a separate bounded ready/valid construct and remains supported.
- Unsupported combinations fail with a diagnostic. Generated signal names are not a source-language interface.

18.2 Smallest valid modules

For buffered streams and numerical components, see stream composition, CRC and FIR: `std.stream.composition` provides `RvFork2`/`RvJoin2`, `std.coding.crc` provides `RvCrc32Byte`, and `std.dsp.fir` provides exact and explicitly quantized parallel FIR. FIR has one elastic output stage, II=1, immutable coefficients, and history that advances only on `input.transfer`.

`std.dsp.nco`, `std.dsp.nco_linear`, `std.dsp.cordic`, `std.dsp.cic`, `std.dsp.polyphase`, `std.dsp.halfband` and coding blocks provide explicit accepted-sample phase, decimation/interpolation, wrap/quantization and bit-order contracts. These are source-authored stdlib components, not new language syntax. For vectoring CORDIC and atomic FIR reload and one-MAC folded FIR, use their separate numerical, coefficient-epoch and latency/II contracts.

Additional source-authored modules:

- SECDED: K=1…64, single-bit correction, double-bit detection, one elastic stage.
- Farrow: M=0…3, K=1…8, raw unsigned μ, exact stored coefficients, M+1 edges and optional final-only quantization.
- DVB RS(204,188): framed systematic encoder and errors-only decoder, up to eight erroneous symbols; status before payload.
- K=7 Viterbi: hard/3-bit-soft pairs, full-frame traceback, deterministic ties, no assumed zero final state or implicit tail bits.

These limits describe the source contracts, not universal proof coverage or the contents of an older installed package. See each guide's status and example.

Combinational logic:

```
module Add {
    in a, b : u8
    out y : u9 = a + b
}
```

Sequential logic:

```
module Accumulator {
    clock clk reset rst
    in enable : bit
    in value : u8
    out total : u9

    reg state : u9 = 0
    update: when enable {
        state <- truncate<9>(state + extend<9>(value))
    }
    total = state
}
```

All operands are read from one pre-edge snapshot. An accepted action group commits atomically; reset has priority. Multiple conflicting actions require an explicit `priority` relation or are rejected.

Multiple domains use the same declarations with one `@clock` owner:

```
module DualClock {
    clock a reset ra @a
    clock b reset rb @b
    reg left : u32 @a = 0
    reg right : u32 @b = 0
    rule TickLeft @a when 1 { left <- truncate<32>(left + 1) }
    rule TickRight @b when 1 { right <- truncate<32>(right + 2) }
}
```

An omitted owner is accepted only when exactly one domain can be inferred. Dynamic provenance follows through local expressions; an explicit `sync_level`, `pulse_toggle`, `handshake`, or `async_fifo(N)` is the only way to cross domains.

18.3 Types, literals, and exact conversions

Common scalar types are `bit`, `u8`, `s16`, `uint<N>`, `sint<N>`, and `bits<N>`. `char` is canonical `u8`; `string<N>` is canonical `vec<N,u8>`. Fixed point uses `fixed<W,F>`, `ufixed<W,F>`, or concise forms such as `SF8.8` and `SF_Sat8.8`.

- Unsuffixed integer literals use minimum exact width unless they occur directly at a concrete typed boundary.
- `a + b` and `a - b` follow the documented exact result-width rules; multiplication returns the full product width. Shifts preserve left width.
- Use `extend<N>` and `truncate<N>` for width changes.
- `quantize<T>` provides explicit fixed-point rounding/overflow. Its placement is part of the computation and cannot be moved into products or reductions.
- Compile-time `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt` generate constants only; non-integral results require explicit `quantize<T>`.
- Use `bitcast<T>` only for equal-width representation changes. `pack`/`unpack` are supported low-level compatibility forms.
- `concat(a,b,...)` places the first operand at the MSB. Destination context never changes operand widths.
- Packed bit index zero is the LSB. `x[MSB:LSB]` uses an inclusive static slice; vector ranges `values[first..past_last]` are half-open.
- Indexed aggregates are packed LSB-first: `vec`/`string` element zero and tuple `item0` occupy the least-significant component recursively. First-declared struct fields and tagged-union tags remain at the MSB.

See types and numerics before changing a width or a fixed-point type.

18.3.1 Operators at a glance

- Arithmetic: `+`, binary `-`, `*`, unary `-`. Built-in integer families do not mix signed and unsigned operands implicitly. For example, `u8 + u8` is `u9`, `u8 - u8` is modular `u8`, `s8 - s8` is `s9`, and `s8 * s8` is `s16`. Assignment does not narrow any of these results.
- Bitwise: `&`, `|`, `^`, `~`; logical `!` requires `bit`. Runtime Boolean control combines `bit` values with `&` and `|`, not `&&` and `||`.
- Shifts: `<<`, `>>` preserve the left operand's width and require an unsigned shift amount. Right shift of a signed integer is arithmetic.
- Comparisons: `==`, `!=`, `<`, `<=`, `>`, `>=` return `bit`. Ordering is numeric; raw bits and enums do not acquire integer ordering implicitly.
- Selection: `condition ? a : b`, `mux`, `switch`, and tagged-union `match`. Runtime conditions are `bit`; branches must have one exact result type.
- `/` and integer `%` are compile-time only. `&&` and `||` belong to the bounded compile-time/guard sublanguages, not runtime hardware truthiness.

See operator precedence and typing for mixed widths, fixed-point rules and overload resolution.

18.4 Aggregates and functions

```
struct Pair<type T> { left : T right : T }

fn duplicate_sum<type T>(a : T, b : T) {
    exact = a + b
    Pair { left = exact right = exact }
}
```

Function bodies are pure and combinational. Bare body bindings are immutable, inferred, declaration-ordered locals; the final expression is the result. Return types may be explicit or inferred. Functions cannot capture module ports or contain state, protocols, delays, or pipelines.

Supported aggregates include `vec<N,T>`, nominal `struct`, nominal `enum`, tagged `union`, and structural tuples `(T,U)`. Prefer inferred locals when the type is exact and obvious. Keep explicit types at public ports, state, raw/enum boundaries, narrowing, and fixed-point quantization.

Struct updates use `value with { field = replacement }`, not a second struct constructor. Enum backing may be `bit`, `bits<W>` or an unsigned integer type. Omitted member codes start at zero or increment the preceding explicit code. Use `Enum.Member`, exhaustive `switch`, and `enum_encode` for a numeric boundary; raw encodings use `enum_valid<T>` and total `enum_decode<T>` with a same-enum fallback. A raw-bit encoding needs an explicit numeric `bitcast` before arithmetic. Top-level enum inputs and arbitrary `unpack<Enum>` are not supported. Enum members such as `reserved`, `reset` and `priority` are valid contextual identifiers; lexical highlighting is not a reserved-name rule.

Generics are monomorphized. Type/value parameters and exact compile-time constraints are source declarations, not runtime hardware:

```
module Queue<type T,D=4> where D >= 2 && is_power_of_two(D) {
    // ...
}
```

Integer `%` is compile-time only: it may appear in generic arguments, static indices and compile-time `if`. It has the precedence of multiplication and division and uses floor modulo (`-5 % 3 == 1`, `5 % -3 == -1`). An evaluated zero divisor is an error. Ports, registers and ordinary function arguments are not compile-time operands; there is no runtime remainder. See the complete contract and PSS example.

Imports are logical. For example `import std.bus.reg` resolves compiler-shipped `stdlib/bus/reg.zhl`; `import std.math.complex as cx` adds a source-local qualifier but does not create a runtime namespace.

18.5 State, hierarchy, and protocols

- Declare one default synchronous domain with `clock clk reset rst`.
- `async reset arst @clk` means asynchronous assertion and fixed two-edge synchronized release. Reset modes are expressed by the clock/reset declarations rather than user logic.
- Add `release externally_synchronized` only when deassertion is already synchronized outside; no internal 2FF is emitted.
- `reg q:T = VALUE` resets to `VALUE`; `reg q:T` has no reset branch.
- `out q:T` is a combinational output wire.
- `out reg q:T [@clock] [= VALUE]` is real architectural state: it holds when not updated and follows register clock/reset scheduling. It is not Verilog's historical `output reg` declaration category.
- `drive q = expression` is a transient rule/FSM-owned logical wire effect, zero when no owning action fires. It is not tri-state, high-impedance, electrical-strength, pad, analog, or multiple-driver-resolution behavior.
- `reg`, `fifo`, `memory`, and `rom` retain explicit typed state/storage semantics. Memory latency, collision, mask, and reset policies are not inferred.

- A concise child declaration is `child : ChildModule`; `inst child : ChildModule` remains accepted. Bind only typed ports/endpoints.
- `rv<T>`, `credit<T,N>`, and `request_response` properties such as `.transfer` are typed observations, not ordinary struct fields.
- Use explicit `connect`/`source -> sink`, adapters, and CDC crossings. The compiler never inserts these silently.
- Top-level structs and tuples become recursively named leaf ports; vectors use multidimensional packed arrays in the production Direct-SV ABI.

Read sequential state/storage and hierarchy/protocols before composing stateful children.

18.6 Functional datapath and implementation intent

`generate`, `map`, `reduce`, `sum`, and `dot` are statically bounded functional hardware expressions. They are not runtime loops.

Common built-ins by purpose:

- Width/representation: `extend`, `truncate`, `bitcast`, `pack`, `unpack`, `concat`, `reshape`, `zeros<N>`, `ones<N>`, and `parity`.
- Fixed point: `quantize<T>`, `fixed_raw<T>`, `fixed_to_raw`, and the explicit `fixed_truncate_*` / `fixed_round_even_*` compatibility conversions.
- Enum boundaries: `enum_encode`, `enum_valid<T>`, `enum_decode<T>`.
- Compile-time shape: `length`, `floor_log2`, `ceil_log2`, `index_width`, `is_power_of_two`; compile-time numerical constants also use `pi`, `sin`, `cos`, `log2`, `log`, `exp`, and `sqrt`.
- Collections: `generate`, `map`, `reduce`, `sum`, `dot`, and `repeat`.
- Timing: `delay<N>(value)` and `pipeline(N) { value }` add exact cycles; they are not ordinary pure-function calls or architecture-search requests.

These are context-sensitive operations, not a promise of arbitrary operand types. Use the linked type/expression chapters for exact signatures. Imported stdlib helpers and user-defined functions are separate from compiler intrinsics.

Use exactly one `implement` region when the value is fixed and the compiler may choose among legal implementations:

```
y = implement {
    dot(samples, coefficients)
    intent {
        latency <= 4
        ii == 1
        dsp <= 8
        fmax >= 100
        minimize lut
    }
}
```

An Fmax constraint or objective compares available implementation choices. It is not a guarantee of achieved device or silicon frequency; estimates, synthesis measurements, and routed measurements are distinct.

The expression is the hardware meaning. `intent` contains hard constraints and one supported objective; it does not name optimization passes. Search is bounded and deterministic rather than an exhaustive optimality guarantee. Positive-latency choices require a valid clock/reset context. `choice` is reserved for user-authored equivalent alternatives. Project profiles may set the documented `intent-exploration` count limits; there is no source-level timeout.

See implementation intent, frequency, and II for every accepted metric/objective, synthesis clock control, project-profile configuration, and the exact ready/valid II>1 boundary.

18.7 Verification

```
contract arithmetic @ clk {
    require legal_inputs { (a < 8) & (b < 8) }
    ensure exact_output { y == a + b }
    cover maximum { y == 14 }
}
```

Named `assert`/`ensure` are same-cycle safety goals. `require` is an environment-owned precondition scoped to its contract. `cover` is bounded reachability; a missed cover is not an unreachability proof. BMC produces `bounded_pass`, never `proven`. Verification syntax does not legalize invalid hardware or feed optimizer range inference.

Liveness, arbitrary SVA/SMT, temporal sequences, and source-level equivalence controls are outside this syntax. Start from the existing formal examples.

18.8 Commands and completion check

```
zlang source.zhl --check
zlang source.zhl --top Top --systemverilog build/Top.sv
zlang source.zhl --top Top --target sky130-fd-sc-hd --systemverilog build/Top.sv
zlang source.zhl --top Top --verify
zlang sim source.zhl --top Top --clock clk --cycles 100
zlang sim source.zhl --top Top --logic-state --clock clk --cycles 100
zlang sim source.zhl --top Top --strict-uninitialized --clock clk --cycles 100
zlang verify build/verify --mode bmc --depth 20 --work-dir build/verify-work
```

Ordinary project compilation validates the checked-in lock read-only. Every `zlang.toml` project requires `zlang.lock`, including zero-dependency projects. Restore the checked-in lock when cloning; for a new project without one, create it explicitly with `zlang lock update --project zlang.toml`. Thereafter run that command only after dependency declarations, dependency source contents, exact Git revisions or other resolution inputs change, not before each simulation or RTL build. Compilation never fetches or rewrites the lock. Standalone sources without a manifest need no lock. See project setup.

In a `--events` JSONL schedule, `"repeat": N` repeats one event lazily for a bounded positive number of steps (at most 1,000,000 expanded events) without a project-specific generator.

Use `--cycles` only for a one-clock top that does not need an explicit reset sequence. Resettable or multi-clock designs should use JSONL so pin levels, inputs and coincident edges are reproducible:

```
{"reset":{"rst":true},"edges":["clk"]}
{"reset":{"rst":false},"set":{"enable":1},"edges":["clk"]}
{"set":{"enable":0},"edges":["clk"],"repeat":4}
```

Run it with `--events events.jsonl`; add `--trace build/run.fst` and repeated `--trace-signal NAME` options for a selective waveform. Add `--compare-with verilator` or `--compare-with iverilog` to replay the same events against generated Direct-SV. Any mismatch, unresolved RTL output, tool failure, or timeout is an error, not a warning.

Ordinary native simulation retains its deterministic two-state zero seed. `--logic-state` enables per-bit `0`/`1`/`U`/`X`: unreset state starts as `U`, an accepted unresolved write becomes `X`, and controlling values such as `U | 1` still resolve exactly. `--initial-reg` overrides selected initial registers and `--trace-signal` selects waveform signals. Prefer `.fst` for native interactive traces; `.vcd` remains supported for interchange. Hidden `__zlang_meta/*_u_mask` traces distinguish `U` from the formats' shared `x` spelling. `--strict-uninitialized` implies this mode and rejects observed `U` or `X`. Simulation executes the selected implementation in both modes. U/X tags are diagnostics, not physical values; legal binary-equivalent optimization may change their propagation. Strict mode checks typed observations and stateful controls, not every syntactic operand in the original expression. Traces and `get_logic()` may retain unresolved tags for inspection. Neither mode models `Z`, electrical resolution, or Verilog event regions. CLI FST/VCD output is an event-index

visualization: physical clock/reset levels match each compiled domain, but the JSONL schedule does not define analog time.

Font license notice

Bitstream Vera / DejaVu fonts

Copyright (c) 2003 by Bitstream, Inc. All Rights Reserved. Bitstream Vera is a trademark of Bitstream, Inc. DejaVu changes are in public domain.

Permission is hereby granted, free of charge, to any person obtaining a copy of the fonts accompanying this license (”Fonts”) and associated documentation files (the ”Font Software”), to reproduce and distribute the Font Software, including without limitation the rights to use, copy, merge, publish, distribute, and/or sell copies of the Font Software, and to permit persons to whom the Font Software is furnished to do so, subject to the following conditions:

The above copyright and trademark notices and this permission notice shall be included in all copies of one or more of the Font Software typefaces.

The Font Software may be modified, altered, or added to, and in particular the designs of glyphs or characters in the Fonts may be modified and additional glyphs or characters may be added to the Fonts, only if the fonts are renamed to names not containing either the words ”Bitstream” or the word ”Vera”.

This License becomes null and void to the extent applicable to Fonts or Font Software that has been modified and is distributed under the ”Bitstream Vera” names.

The Font Software may be sold as part of a larger software package but no copy of one or more of the Font Software typefaces may be sold by itself.

THE FONT SOFTWARE IS PROVIDED ”AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL BITSTREAM OR THE GNOME FOUNDATION BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

Except as contained in this notice, the names of Gnome, the Gnome Foundation, and Bitstream Inc., shall not be used in advertising or otherwise to promote the sale, use or other dealings in this Font Software without prior written authorization from the Gnome Foundation or Bitstream Inc., respectively. For further information, contact: fonts at gnome dot org.